



FIWARE IoT agent for Matter: Toward the integration of smart home devices into the FIWARE smart city platform

Nguyen Thi Dieu Linh^a, Thanh-Trung Nguyen^{b,c,*}

^a Department of Science and Technology, Hanoi University of Industry, Hanoi, Viet Nam

^b Intelligence Data Processing System Research Group, Dong Nai Technology University, 206 Nguyen Khuyen Street, Trang Dai Ward, Dong Nai, Viet Nam

^c Faculty of Information Technology, Dong Nai Technology University, 206 Nguyen Khuyen Street, Trang Dai Ward, Dong Nai, Viet Nam

ARTICLE INFO

Keywords:

FIWARE

Matter

FIWARE IoT agent for matter

Smart home x smart city

ABSTRACT

The rapid adoption of the Matter protocol as a global standard for smart home appliances presents a significant opportunity to enhance the interoperability of IoT devices. Concurrently, FIWARE, a widely adopted smart city platform, facilitates seamless integration and management of diverse IoT devices through standardized data models and context management capabilities. However, integrating Matter-enabled devices with FIWARE remains a challenge due to differences in protocol architectures and data representations. This paper proposes a novel framework to bridge the Matter protocol and the FIWARE platform, enabling interworking between smart home appliances and smart city services. The proposed solution includes: (1) a data model and adaptive translation algorithm for mapping Matter device specifications to FIWARE Smart Data Models, (2) an IoT Agent to interface Matter devices with the FIWARE platform as FIWARE resources, and (3) a comprehensive evaluation using commercial and experimental Matter-enabled devices. This approach ensures seamless data exchange, enhances cross-domain interoperability, and supports the integration of Matter devices into smart city contexts. The results demonstrate the feasibility and efficiency of the integration, offering a scalable framework for leveraging low-cost Matter devices as assets within smart city platforms. This work contributes to advancing IoT interoperability and paves the way for innovative applications in smart homes and smart cities.

1. Introduction

The Internet of Things (IoT) is transforming urban living by driving the development of smart cities. By integrating IoT technologies, cities are becoming more efficient, sustainable, and responsive to citizens' needs [1]. This transformation optimizes resource management, enhances public services, and improves overall quality of life. Through real-time data collection and analysis, smart city initiatives support informed decision-making in critical areas such as traffic management, energy consumption, and public safety. As cities become increasingly interconnected, residents benefit from seamless technology integration, leading to greater convenience, safety, and sustainability [2].

To enable this transformation, significant efforts have been made to develop smart city platforms — often referred to as smart city middleware [3] or operating systems — which form the foundation of an interconnected urban ecosystem. These platforms facilitate the horizontal integration of diverse domains, ensuring seamless data exchange and interoperability across sectors. Among them, the FIWARE [4] framework has demonstrated its effectiveness in enabling large-scale

smart city deployments. By ensuring seamless communication and interoperability among diverse IoT devices and platforms, FIWARE plays a crucial role in federating pilot projects across America, Europe, and Asia, supporting the successful implementation of large-scale smart city projects [5].

1.1. Motivation

The FIWARE framework is well-known for its modular architecture, which allows for easy plug-and-play integration of data processing, analysis, publication, and interfaces that connect IoT, robotics, and third-party systems such as port management [6], agriculture [7], and water management [8]. However, integrating smart homes into the broader smart city landscape is not yet fully achievable within FIWARE. Smart homes are no longer a new concept—they have been extensively implemented over the past decade. The technology has matured as society increasingly prioritizes convenience and comfort [9]. Connecting smart home data to smart city infrastructures has a significant

* Corresponding author at: Faculty of Information Technology, Dong Nai Technology University, 206 Nguyen Khuyen Street, Trang Dai Ward, Dong Nai, Viet Nam.

E-mail addresses: nguyen.linh@hau.edu.vn (N.T. Dieu Linh), nguyenthanhtrung@dnvu.edu.vn (T.-T. Nguyen).

<https://doi.org/10.1016/j.comcom.2025.108369>

Received 24 February 2025; Received in revised form 17 November 2025; Accepted 18 November 2025

Available online 19 November 2025

0140-3664/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

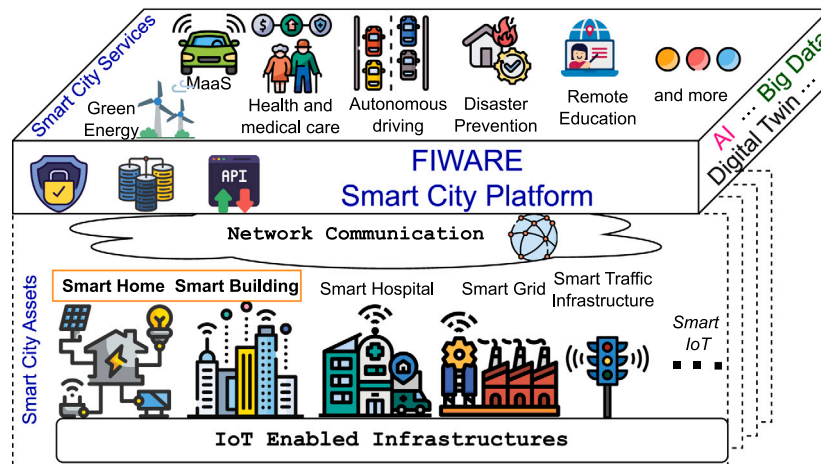


Fig. 1. FIWARE provides a unified smart city platform designed to interconnect smart city assets with smart city services. It offers a range of modules for big data processing, AI/ML model integration, and robust security measures. This work focuses on integrating Matter, a widely adopted protocol for devices in the smart home and smart building domains, into the FIWARE ecosystem.

impact on developing sustainable systems. Micro-level residential data can improve predictive analytics, such as forecasting traffic patterns or energy consumption [10]. Additionally, smart home integration can enhance quality of life [11], particularly for elderly individuals, by enabling them to live more independently [12,13]. In the event of a disaster, data from smart homes can assist in creating hazard maps and expediting evacuation processes, ultimately improving emergency response efforts (see Fig. 1).

In this work, we propose and implement a solution to integrate Matter, an emerging smart home protocol, into the FIWARE framework, unlocking the full potential of smart homes within the broader smart city ecosystem. Matter¹ is designed to establish a standardized interface for smart home appliances, attracting widespread global attention [14]. Backed by major consumer electronics companies such as Google, Apple, Amazon, and Samsung, it is widely regarded as the future standard for smart homes [15]. Since its official introduction at CES 2023, numerous vendors have adopted Matter in their products, further cementing its role in the smart home landscape [16].

Due to the growing popularity of the Matter protocol, its integration into IoT platforms like FIWARE is in high demand, creating a win-win scenario for both hardware vendors and IoT platforms. For hardware vendors, Matter-compliant products gain broader applicability across various domains, increasing their impact. Meanwhile, IoT platforms benefit from a vast influx of connected devices, providing rich data to enhance and develop better services.

At the time of writing, this work is the first to propose and implement the integration of Matter with FIWARE.

1.2. Position of this work in the ITU-T Y.2070

The ITU-T Y.2070 [17] provides a framework for the architecture, terminology, and key components for the deployment and operation of IoT systems in smart home environments. As this work focuses on smart home systems, we align its components with the ITU-T Y.2070 architectural framework to ensure coherence and facilitate understanding, as outlined below.

- **ITU-T Y. 2070 Application Layer | Future development:** Smart smart home, smart city related services that can be developed thanks to this work.

- **ITU-T Y. 2070 Platform Layer | FIWARE Platform:** The FIWARE Context Platform serves as the common infrastructure, enabling interoperability across different domains.
- **ITU-T Y. 2070 Gateway Layer | This Work:** The FIWARE IoT Agent for Matter in this work that connects Matter devices (Device layer) to FIWARE (Platform layer).
- **ITU-T Y. 2070 Device Layer | Matter:** Matter provides a standardized language that ensures interoperability among devices from different vendors.

1.3. Contribution

The primary challenge in integrating Matter with FIWARE lies in how to represent and synchronize Matter devices as resources within FIWARE, while facilitating interaction between FIWARE-enabled services and devices. This work presents a detailed solution to address these gaps. Our contributions are summarized as follows:

- Propose an algorithmic solution to automatically generate a data model for representing Matter devices in a FIWARE-compliant format, thereby ensuring semantic interoperability with the existing FIWARE framework. This data model facilitates seamless interaction with Matter devices, eliminating the need for FIWARE developers to have prior knowledge of the Matter specification. The data model serves as the core component of the Matter-FIWARE integration.
- Design and implement a software solution that discovers and exposes Matter devices as FIWARE resources, enabling smooth interaction between FIWARE services and Matter devices. When deployed on hardware such as a Raspberry Pi, this solution acts as a Matter controller, akin to commercial products like Google Nest Hub or Apple HomePod, which connect Matter devices to their respective ecosystems. However, our solution is not intended to replace these existing platforms but rather serves as an additional device with unique features, extending the capabilities of Matter devices beyond proprietary ecosystems and enhancing their interoperability within smart city infrastructures.
- Conduct experiments to validate and evaluate the proposed solution using both commercial and experimental Matter devices, in conjunction with the FIWARE Orion Context Broker.

1.4. Article structure

The structure of this article is as follows: Section 2 provides an overview of the fundamental concepts of Matter and the FIWARE

¹ <https://csa-iot.org/all-solutions/matter/>

framework. Section 3 reviews related work on both protocols. Section 4 presents an overview of our proposed approach. In Section 5, we introduce a solution for generating the data model, while Section 6 details the design, required components, and core interactions of the Matter-FIWARE controller. Section 7 covers the implementation, experimentation, and evaluation of our solution. Section 8 provides discussion and potential usecases. Finally, Section 9 summarizes our contributions and findings.

2. Background

This section provides a concise overview of the technologies we intend to utilize, focusing on the Matter protocol and the FIWARE framework. Additionally, it emphasizes their data models, which are critical for developing a standardized approach to describing physical devices. These data models serve as the foundation for enabling seamless mapping of device data between the two frameworks, facilitating interoperability and bidirectional data exchange.

2.1. FIWARE platform

FIWARE originated in 2011 as a flagship initiative of the European Union's Future Internet Public-Private Partnership (FI-PPP) program, aiming to establish an open, sustainable ecosystem for developing smart applications and services. Its mission was to democratize innovation by providing universal standards and tools for digital solutions. Over time, FIWARE evolved into a global enabler of interoperable smart city platforms and cross-domain systems, adopted not only across Europe but also in regions like Japan, North America, and beyond [18].

The overall architecture of the FIWARE framework, as of release 8.5.1, is summarized in Fig. 2. At its core, FIWARE is built around context brokers that support both NGSI-v2 and NGSI-LD standards. These brokers enable vertical integration by connecting devices, sensors, and services, such as linking traffic lights to a city's mobility management system, and horizontal integration by bridging diverse domains, including energy, transportation, and healthcare, to support holistic urban planning. Surrounding the context brokers are a set of Generic Enablers and Incubated Enablers, which provide additional functionalities such as data storage, processing, analytics, and visualization.

On the southbound interface, FIWARE includes enablers specifically designed to connect IoT devices, robotics platforms, and third-party systems, ensuring seamless data ingestion from the physical world. Beyond connectivity, FIWARE offers comprehensive frameworks for data security, governance, publication, and monetization, thereby allowing stakeholders to customize solutions to their specific requirements. This modular and extensible architecture enables cities, enterprises, and developers to design scalable, interoperable, and future-proof smart systems [19].

In the scope of this work, we focus on developing an IoT agent that connects Matter devices to the FIWARE core context brokers so that current power-by-FIWARE services can utilize data from Matter devices to build better services. In FIWARE, every device is represented as an Entity with associated Attributes. The detail of the FIWARE data model will be described in the next section.

The CRUD operations of entities at the context broker are predefined by FIWARE. Actions and corresponding HTTP requests are summarized in Tables 1 and 2.

The following tables summarize the actions required to perform CRUD operations on an entity or its attributes. For instance, in Table 1, the IoT agent managing a Matter device typically creates an entity representing the physical device in the broker using an HTTP POST operation. A service can then use an HTTP GET request to retrieve the entity that describes the Matter device from the broker. Similarly, in Table 2, a service can control the device by updating the value of the corresponding entity. The IoT agent will then process the update and apply the command to the device. The context broker effectively

decouples services and devices, enabling interoperability, scalability, and real-time interaction between devices and services.

For instance, as shown in Table 1, the IoT agent first registers and makes an IoT device available via the broker using an HTTP POST request. Services and applications can then retrieve this entity's data via an HTTP GET request, enabling real-time monitoring of the device's status. To control the device, a service updates the corresponding entity's attributes using an HTTP PUT request, as described in Table 2.

This decoupled architecture, where services interact solely with the broker rather than directly with the device, ensures interoperability (services do not need to understand IoT device's protocol), scalability (thousands of devices can be managed uniformly), and real-time interaction (state changes propagate instantly across the ecosystem).

2.1.1. FIWARE smart data models

Data models are fundamental to FIWARE, enabling interoperability and seamless integration across diverse systems to ensure standardized data exchange and interpretation among IoT devices and services from different manufacturers. Since FIWARE operates at the platform layer, its data models are designed to be service-oriented and adhere to the Next Generation Service Interface (NGSI) specifications for defining data structures.

As illustrated in Fig. 3, FIWARE Smart Data Models support both NGSI-v2 [20] and NGSI-LD [21]. NGSI-V2 focuses on simplicity and ease of use, while NGSI-LD builds upon NGSI-V2 by incorporating linked data principles to represent data as a semantic graph, enhancing semantic interoperability across smart systems.

- NGSI-v2, an entity represents a real-world object (e.g., a light bulb or door lock) with associated attributes (e.g., currentStatus, brightness), which may include metadata such as measurement units (e.g., lux or kilolux). Each entity has a unique id and a type (e.g., "Light").
- NGSI-LD represents an evolution of the NGSI-v2 information model, specifically enhanced to support linked data, property graphs, and semantic representation through JSON-LD. Its core constructs are entity, property, and relationship. An NGSI-LD entity instance can be associated with properties or relationships. In NGSI-v2 terms, a Property corresponds to an attribute-value pair, while Relationships establish associations between instances using linked data principles.

Although NGSI-v2 and NGSI-LD differ in how they represent data, both are centered around entities with associated attributes or properties and their corresponding values. For simplicity, this work adopts an NGSI-v2 data model to describe Matter devices.

2.2. Matter smart home protocol

Matter is an IPv6-based protocol operating at the application, presentation, and session layers (OSI Layers 5–7) [22], with its protocol stack (Fig. 4) embedded in every Matter-compliant device to ensure standardized interoperability. The stack includes the Bluetooth Transport Protocol (BTP) for seamless device commissioning via Matter-over-Bluetooth Low Energy (BLE), enabling secure onboarding of devices into a network. For data transmission, Matter enhances UDP through its Matter Reliability Protocol (MRP), which incorporates retransmission mechanisms, delivery confirmations, and duplicate message detection to improve reliability in loss-prone environments. At the heart of the Matter specification lies its communication middleware, which defines interaction models, message framing, and security mechanisms (e.g., encryption, authentication) to ensure secure, standardized device communication. This middleware serves as the protocol's backbone, governing how data is packaged, transmitted, and protected across networks. Complementing this is the data model, a critical component that standardizes the semantics of devices (e.g., defining what a

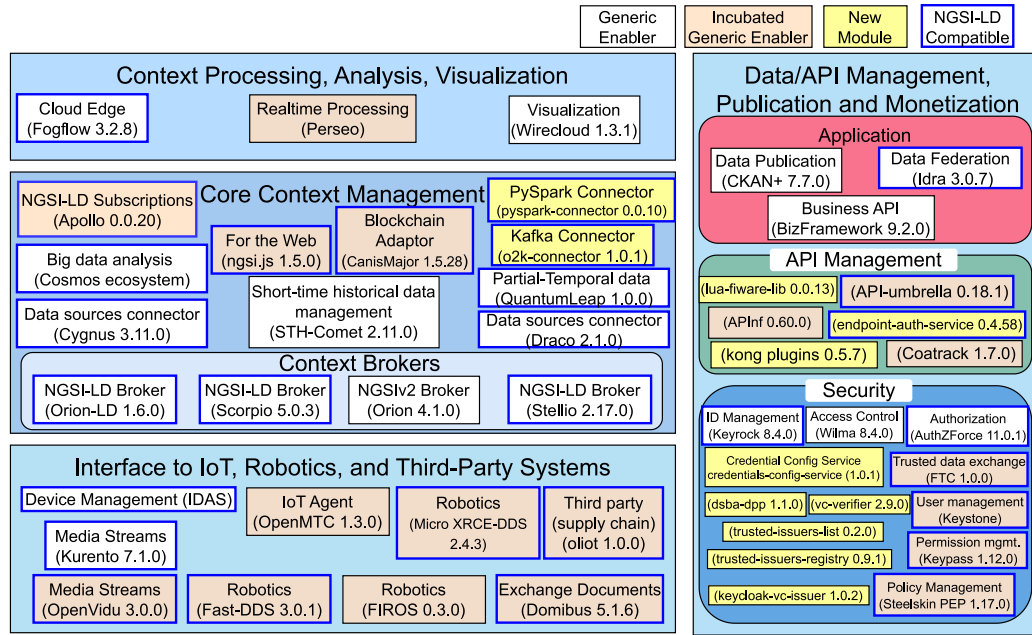


Fig. 2. Overview of the FIWARE framework components in release 8.5.1.

Table 1
CRUD operations for entity.

HTTP	/v2/entities	/v2/entities/entity-id
POST	CREATE a new entity and register it with the broker	CREATE or UPDATE an attribute of an entity
GET	READ data from all entities registered at the broker	READ data from an entity
DELETE	Not Available	DELETE an entity from the broker

Table 2
CRUD operations for attribute.

HTTP	.../attrs	.../attrs/{attribute-name}	.../attrs/{attribute-name}/value
GET	Not Available	Not Available	READ value of an attribute from an entity
PUT	Not Available	Not Available	UPDATE value of an attribute from an entity
PATCH	UPDATE value of one or more attributes from an entity	Not Available	Not Available
DELETE	Not Available	DELETE an attribute from an entity	Not Available

“dimmable light” is, its capabilities, and how to control its brightness levels). By providing a universal vocabulary for vendors to describe their products, the data model ensures interoperability across products from different vendors. The specifics of this model, including its hierarchical structure and attribute definitions, will be explored in later sections. Finally, the Application Layer translates the data model into actionable behaviors, enabling devices to execute real-world functions (e.g., adjusting lighting, managing thermostat settings) based on standardized commands. Together, these layers form a cohesive framework that guarantees consistent, vendor-agnostic operation across Matter-certified ecosystems.

Fig. 5 illustrates a typical smart home setup with Matter devices. There are two main types of Matter devices:

- Native Matter devices – These connect directly using Wi-Fi, Ethernet, or Thread [23], which are natively supported by Matter.

- Bridged devices – These are non-Matter devices that can function as Matter-compatible through a Bridge. The Matter specification also defines the requirements for these bridges.

Currently, the industry follows two main approaches: developing products that natively support Matter or creating bridges to integrate existing non-Matter devices into the Matter ecosystem.

- Matter controllers (or Matter hubs) are responsible for managing a group of devices within their networks, known as Fabrics. The controller plays a crucial role in securely setting up and coordinating Matter devices.

Matter does not specify how IoT platforms should integrate with devices; instead, this is determined by individual vendors. For instance, the Google Matter controller (Google Nest Hub) connects Matter devices within its Fabric to the Google platform, just as Amazon and other

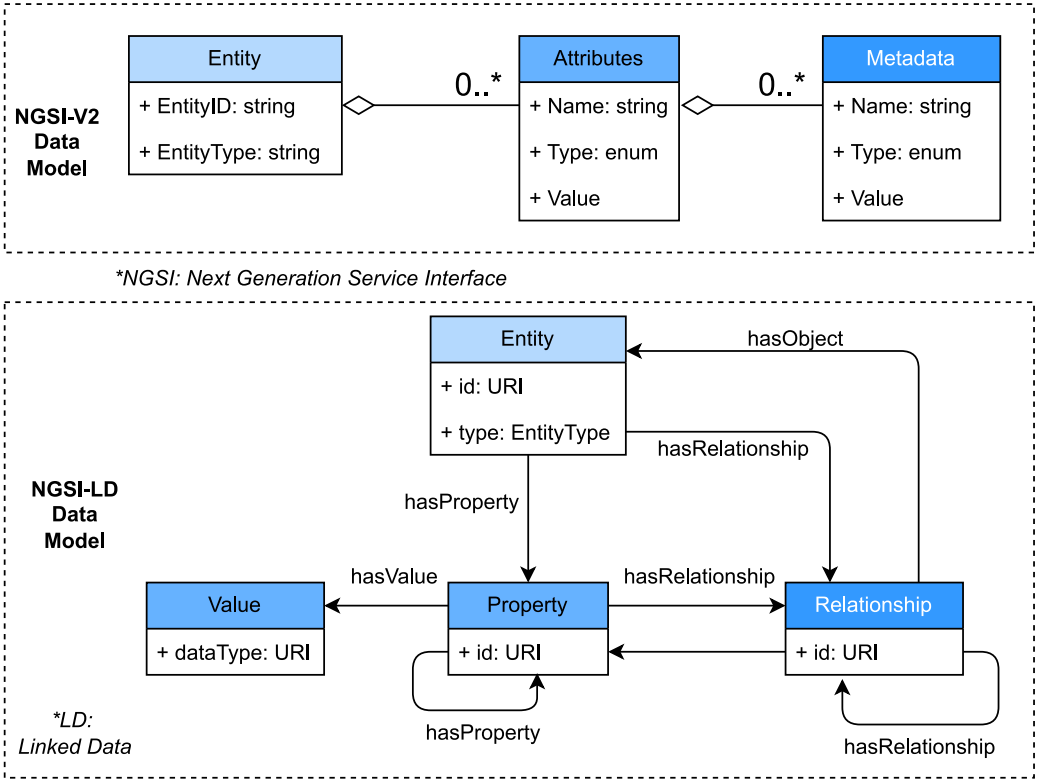


Fig. 3. Matter Protocol Stack within the OSI Model. The communication middleware facilitates device-to-device interactions, while the data model offers a universal vocabulary to describe devices from different vendors in an interoperable manner.

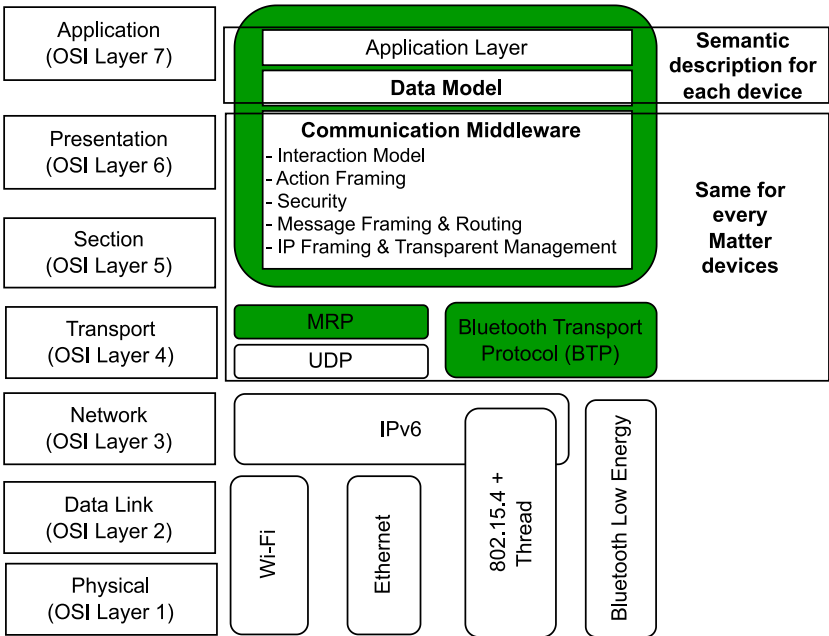


Fig. 4. Matter protocol stack within the OSI Model: The Communication middleware facilitates device-to-device interaction, while the data model provides a universal language for describing devices from different vendors in an interoperable manner.

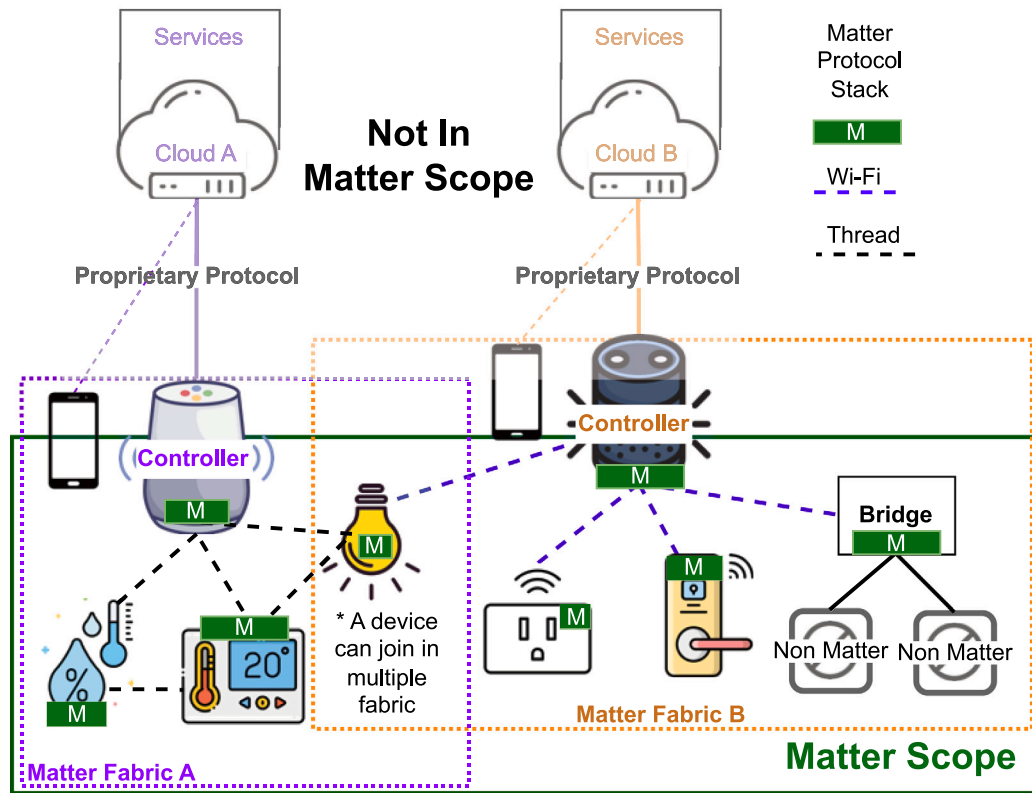


Fig. 5. A typical smart home topology featuring Matter devices. The proposed solution introduces a Matter Controller integrated with the FIWARE platform as its northbound system. The controller supports the multi-fabric concept, ensuring compatibility with existing deployments without disruption.

companies do with their own platforms. One key advantage of Matter devices is their ability to connect to multiple Fabrics simultaneously, allowing integration with platforms like Google, Amazon, and other systems simultaneously. In practice, the actual number of fabrics a device can support depends on its hardware capabilities, particularly its memory and processing resources. For example, discussions in the Matter development community indicate that while supporting 4–5 fabrics is common, supporting up to 16 fabrics may be challenging for devices with limited RAM.² This work introduces a controller that connects Matter devices to an open, standardized FIWARE platform, enabling greater flexibility and interoperability.

2.2.1. Matter data model specification

One key success of Matter is defining a common data model and this data model is accepted by manufacturers, it is the key for other systems to work with Matter. Since Matter is focusing on the device layer, the data model is device-friendly which means they are binary-based data model. Fig. 6 illustrates how a physical Matter device is described. In the Matter specification, each device is represented by a digital object called a node. When a node joins a Matter fabric, it is assigned a unique node ID for management purposes. A node can have multiple endpoints, with a mandatory endpoint 0 that acts as a “readme” endpoint, containing the node’s “profile” information. Other endpoints contain the actual functionality of the device, for example, an endpoint representing the lighting function of a smart light. Each endpoint consists of several clusters, which are groups of related attributes, commands, and events. For instance, an On/Off light endpoint has an On/Off cluster that defines an attribute with code 0x0000 to represent the light’s On/Off status, along with commands to turn the light on, turn it off, or toggle its state.

Matter’s strategy is to define various clusters as modular “building blocks”, similar to LEGO blocks. A node is built by combining multiple clusters to reflect its full functionality, just like assembling LEGO blocks to create a complete model.

In the Matter specifications version 1.4, there are 112 different cluster types [24] used to define 72 types of devices [25]. It is important to note that the Matter data model is only intended for interpreting binary payloads exchanged with Matter devices. To represent Matter device information on the FIWARE platform, an NGSI-v2-based data model for Matter is required.

The mapping principles for this transformation are as follows:

- Matter endpoints are mapped to FIWARE entities.
- Matter clusters are ignored to avoid nested JSON.
- Attributes of each Matter cluster are converted into attributes of the corresponding FIWARE entities.
- Commands of each cluster are mapped to entity operations.
- Events of each cluster are converted into entity attributes with notification capabilities.

3. Related work

In current FIWARE platforms, several Generic Enablers focus on integrating IoT and other systems into FIWARE [26,27]. Table 3 summarizes a non-exhaustive list of IoT Agents that are currently supported and incubated by FIWARE. These efforts aim to attract more data from devices and third-party systems into FIWARE, addressing the chicken-and-egg dilemma: the more data sources FIWARE can incorporate, the more services it can offer. This, in turn, drives the adoption of FIWARE, leading to more devices connecting in order to utilize these services.

In the context of smart homes, the work in [28] presents the idea of integrating a home energy management system into FIWARE. However, the devices in this system do not follow any standard smart home

² <https://github.com/project-chip/connectedhomeip/issues/9261>

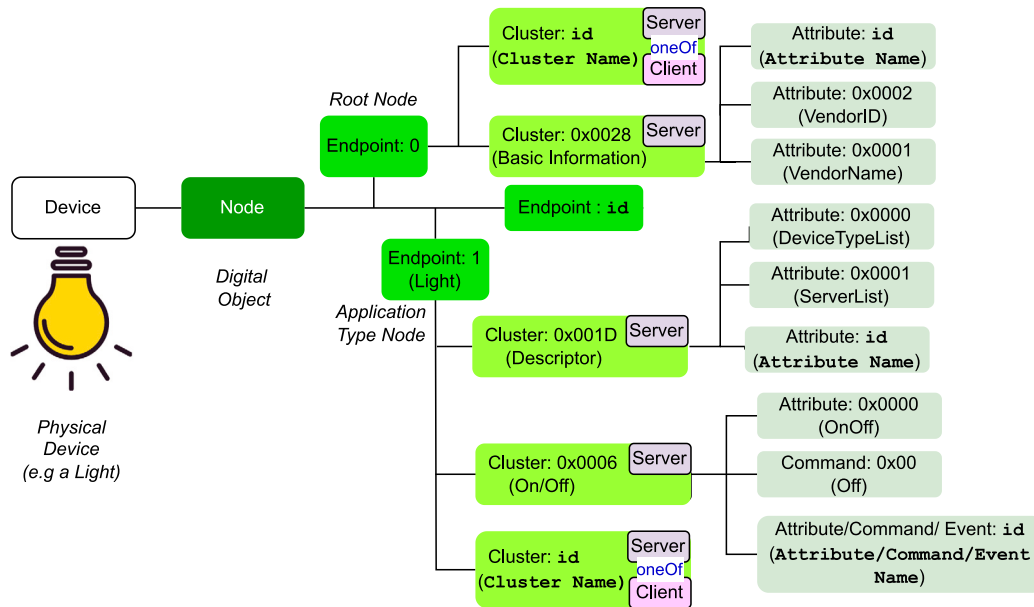


Fig. 6. Overview of the Matter data model.

Table 3
Non-exhaustive list of FIWARE IoT agents.

IoT agent	Description	Protocols
Core Library ^a	Core library to develop IoT agents for FIWARE	Free to extend
for JSON ^b	IoT Agent for a JSON based protocol	AMQP, HTTP, MQTT
for Ultralight ^c	IoT Agent for the Ultralight 2.0 protocol	AMQP, HTTP, MQTT
for LoRaWAN ^d	IoT Agent for the LoRaWAN	LoRaWAN
for Lwm2m ^e	IoT Agent for OMA lightweight M2M protocol	CoAP
for Sigfox ^f	IoT Agent for the Sigfox protocol	Sigfox
for ADAPT/ISOXML ^g	IoT Agent for the ISOXML/ADAPT protocol	ISOBUS

^a <https://github.com/telefonicaid/iotagent-node-lib>^b <https://github.com/telefonicaid/iotagent-json>^c <https://github.com/telefonicaid/iotagent-ul>^d <https://github.com/Atos-Research-and-Innovation/IoTagent-LoRaWAN>^e <https://github.com/telefonicaid/lightweightm2m-iotagent>^f <https://github.com/telefonicaid/sigfox-iotagent>^g <https://github.com/FIWARE/iotagent-isoxml>

protocols, limiting the applicability of this solution to other smart home environments. In [29], the authors propose an IoT agent that integrates ECHONET Lite [30], a dominant smart home protocol in Japan, into FIWARE. However, ECHONET Lite is primarily used in Japan, and the high cost of devices makes this solution less practical for real-world applications.

The need to integrate Matter, a globally recognized protocol for low-cost, widespread smart home appliances, into FIWARE has been a topic of discussion within the community. However, to date, no solution has been introduced.

Matter aims to establish an open, royalty-free protocol and hosts an open-source project on GitHub.³ Table 4 provides a non-exhaustive list of sample projects from the current SDK. To achieve interoperability between Matter and other systems/protocols, the Matter Bridge and Matter Controller are key components to start with.

Most of the current efforts to implement Matter bridges are focused on commercial products. Companies like Hue and Switchbot are actively developing Matter bridges to integrate their existing product lines into the Matter ecosystem. The Matter specification includes a dedicated chapter on bridging mechanisms for incorporating Zigbee or Z-Wave devices into Matter. However, academic research on this topic remains limited. One notable contribution is [31], where the authors propose a bridge to transition Echonet Lite, a widely used smart home protocol in Japan, to Matter.

Similarly, the development of Matter controllers is primarily driven by large ecosystems such as Apple, Google, Amazon, and Samsung SmartThings, which offer commercial Matter controllers to seamlessly integrate Matter end devices into their ecosystems. While there are other efforts, such as the Home Assistant controller⁴ or the work in [32] introducing open-source controllers, however, these solutions

³ <https://github.com/project-chip/connectedhomeip>⁴ <https://www.home-assistant.io/>

Table 4
Non exhaustive list of sample projects from the Matter SDK.

Project name	Type	Description
chip-tool ^a	Matter Controller	A controller application to interact with Matter devices.
bridge-app ^b	Matter Bridge	An example application that acts as a bridge, integrating non-Matter devices into the Matter ecosystem
all-clusters-app ^c	Matter Emulator	A dummy device type that emulates all available clusters of Matter specification
lighting-app ^d	Matter Emulator	A device emulator that simulates the functionalities of a dimmable light
air-quality-sensor-app ^e	Matter Emulator	A device emulator that simulates the functionalities of an air quality monitoring sensor

^a <https://github.com/project-chip/connectedhomeip/tree/master/examples/chip-tool>

^b <https://github.com/project-chip/connectedhomeip/tree/master/examples/bridge-app>

^c <https://github.com/project-chip/connectedhomeip/tree/master/examples/all-clusters-app>

^d <https://github.com/project-chip/connectedhomeip/tree/master/examples/lighting-app>

^e <https://github.com/project-chip/connectedhomeip/tree/master/examples/air-quality-sensor-app>

only manage Matter device within the local network. An attempt presented in [33] proposes integrating Matter into the W3C Web of Things platforms [34] to address vendor lock-in at the platform level. However, this work is still in its conceptual phase. In our previous work [35], we implemented the integration of Matter with oneM2M, an open, general-purpose machine-to-machine platform. Furthermore, in [36], authors proposed and implemented the connection between oneM2M and FIWARE, which could indirectly facilitate the integration of Matter and FIWARE. However, this integration requires at least two proxies or connectors, which could impact the overall cost and performance of the systems.

4. Proposed theory for matter-FIWARE integration

The theory of Matter-Fiware interworking is illustrated as in Fig. 7. The ultimate goal of this work is to ensure that smart home data from Matter appliances is accessible via the FIWARE platform. This enables not only basic home automation services but also more advanced services that leverage broader contextual information for smart homes. To build a Powered-by-FIWARE system, the essential requirement is that information from the target system must be available at the FIWARE Context Broker. Once resources are registered at the broker, other context brokers, FIWARE Generic Enablers, and services can discover and interact with these resources using FIWARE-defined standards or other FIWARE-compliant mechanisms. To achieve this vision, two main tasks must be accomplished:

- **FIWARE Smart Data Model for Matter:** A data model is needed to instantiate resources using data from Matter devices. FIWARE leads a collaborative effort to maintain public, free data models for various domains and protocols, such as robotics, health, energy, and logistics. A new data model focused on Matter specifications is designed and created to represent smart home device information.
- **Matter Controller as a FIWARE IoT Agent:** The Matter controller acts as a FIWARE IoT Agent, managing the Matter fabric. It registers Matter devices in its fabric with the FIWARE Context Broker, synchronizes the latest data from physical Matter devices with their corresponding digital twins (resources at the broker), and forwards control commands from the broker to the physical devices, ensuring seamless interaction and real-time device control.

5. FIWARE smart data model for matter

In most existing approaches, the construction of data models to represent Matter devices on platforms such as those discussed in [33,35] typically involves the manual mapping of relevant Matter concepts to

corresponding concepts within the target platform. In this study, we build upon the contributions of the open-source community, which has digitized the Matter specification by providing a structured representation of the Matter data model in XML format. Leveraging this structured resource, we automate the generation of both NGSI-V2 and NGSI-LD data models for Matter. This approach not only enhances the consistency of the generated models but also mitigates the tedious and error-prone nature of manual data model construction, as highlighted in [37].

5.1. Formalizing matter data model specification

The initial step involves formalizing the data model to enable the XML-based representation data, available through the open-source project's GitHub repository,⁵ to be processed programmatically as software objects. To achieve this, we propose a Unified Modeling Language (UML) diagram to formalize the structure of the Matter data model, as illustrated in Fig. 8.

The UML class diagram provides a structured representation of the Matter device model specification, enabling its transformation into a programmable data model.

- **Device Class:** Represents a Matter application endpoint within a Matter node, serving as the primary entity in the data model. If a single physical Matter device (node) contains multiple endpoints with distinct application types, it is logically divided into multiple separate devices, each corresponding to an individual endpoint. Each device is associated with multiple clusters, which collectively define its functional capabilities and behavior.
- **Cluster Class:** Represents a logical grouping of related functionalities within the **Device** class. Each cluster encapsulates attributes, commands, events, and parameters that define its properties and operations. A key property of this class is `isServer`, which indicates whether the cluster can act as a server, receiving interactions, or as a client, initiating interactions with other clusters. There are two types of clusters: `ApplicationCluster`, which represents the functional components of an actual device, and `UtilityCluster`, which provides profile-related or diagnostic information. While `ApplicationCluster` focuses on the primary operations of the device, `UtilityCluster` supports device management and monitoring functions.
- **Attribute Class:** Represents configurable properties of a cluster, defined by metadata such as id, name, data type, and data constraints. It also includes flags indicating whether the attribute is

⁵ https://github.com/project-chip/connectedhomeip/tree/master/data_model

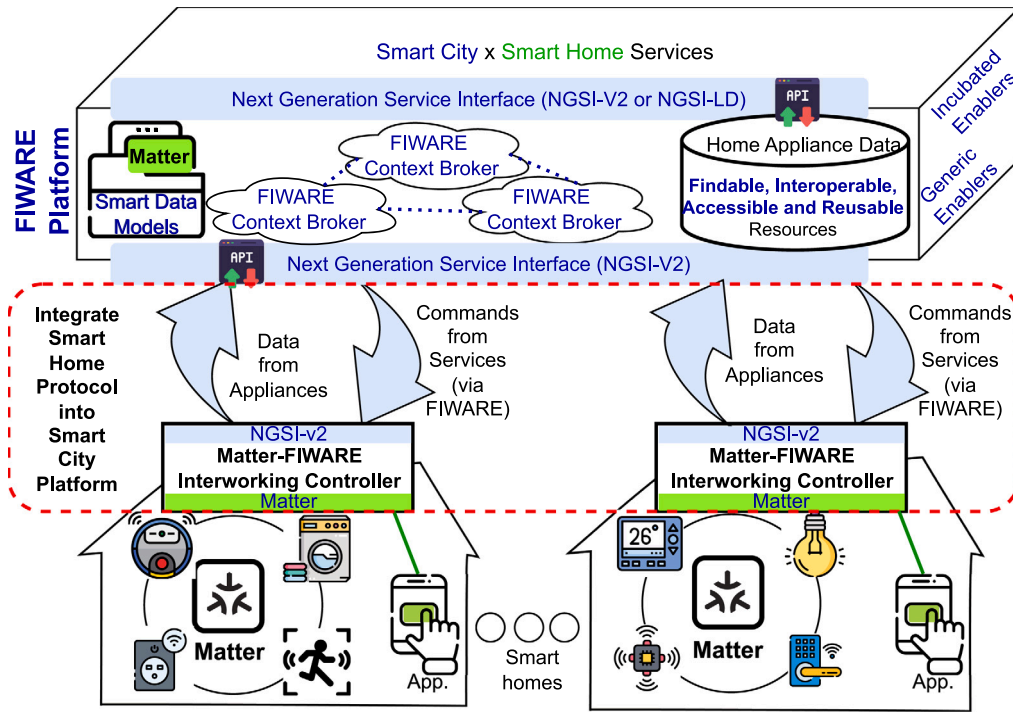


Fig. 7. Conceptual Framework for Matter-FIWARE Integration: (1) a data model for representing Matter device information as resources in FIWARE, ensuring these resources are Findable, Accessible, Interoperable, and Reusable (FAIR) via FIWARE, and (2) a Matter controller functioning as a FIWARE IoT Agent, synchronizing data from appliances to corresponding resources in FIWARE context brokers and applying commands from services to Matter devices, enabling seamless device interaction.

mandatory or if its value can be nullable. Access control indicators specify the attribute's permissions, such as readable, writable, or reportable when the data value changes.

- **Command Class:** Specifies actions associated with a cluster, defined by attributes such as id, name, and a flag indicating whether the command is mandatory. Additionally, it identifies whether the command requires input parameters and provides details for their configuration.
- **Event Class:** Represents significant occurrences or state changes within a cluster, defined by attributes such as id, name, and a flag indicating whether the event is mandatory. It also specifies the data format of the result generated by the event, ensuring accurate interpretation of the event's output.

The next step involves defining serialization rules for the target platforms, specifically NGSI-V2 and NGSI-LD. These rules can also be extended to support additional purposes, such as generating data models for integration with other platforms or constructing an ontology for Matter to enable semantic representation. By generating all data models from a single, unified source, this approach ensures consistency across platforms and use cases.

5.2. NGSI-V2 data model for matter

The algorithm for automatically generating FIWARE data models from Matter specifications is presented in Algorithm 1. Utilizing the UML class diagram described above, we formalize the Matter data model as input to the algorithm. In Line 3, the algorithm iterates over all available clusters associated with a Matter device (endpoint) to extract relevant information. However, as indicated in Line 4, only clusters with the server role are considered. This distinction is necessary because Matter devices may include both devices to be controlled, associated with server clusters, and devices that control

others via client clusters. The latter, which are typically responsible for physical control operations, fall outside the scope of FIWARE integration.

In Line 5 and 6, the algorithm adds a flag to every cluster to identify whether each cluster is an **application** cluster or a **utility** cluster. Only application clusters and a specific subset of utility clusters, namely, the Basic Information Cluster, are converted into FIWARE data models. This cluster provides essential profile metadata such as manufacturer name, software version, and hardware version. Other utility clusters, which serve purposes such as local device management are not included in the FIWARE data model generation, as they are primarily intended for use by controllers within the local Matter network and are not relevant to the external representation in FIWARE.

From Line 8 to 15, the algorithm examines the internal structure of each selected cluster to extract the supported attributes, events, and commands. These elements represent the functional and descriptive capabilities of the device.

From Line 25 to 29, the extracted data from the Matter specification is transformed into NGSI-based data models. While NGSI-V2 and NGSI-LD differ slightly in their formats, particularly in how attributes and commands are appended to entities, the core principles and overall conversion process remain consistent across both formats.

Fig. 9 illustrates an example of mapping the Matter On/Off Light to the corresponding FIWARE OnOffLight entity, following the algorithm described earlier. In Matter, an attribute can be categorized as readable, writable, or reportable (indicating its latest status). However, the NGSI data model in FIWARE—designed for service development—manages these attributes differently.

In NGSI-V2, all entity attributes are both readable and writable by default through predefined actions and URIs, as specified in Tables 1 and 2. If an application wishes to receive notifications about changes in specific attributes (single or multiple), it must create a subscription for those attributes within the entity. Consequently, reportable attributes or events in Matter are represented as standard NGSI attributes in FIWARE. The FIWARE Context Broker receives updates from Matter

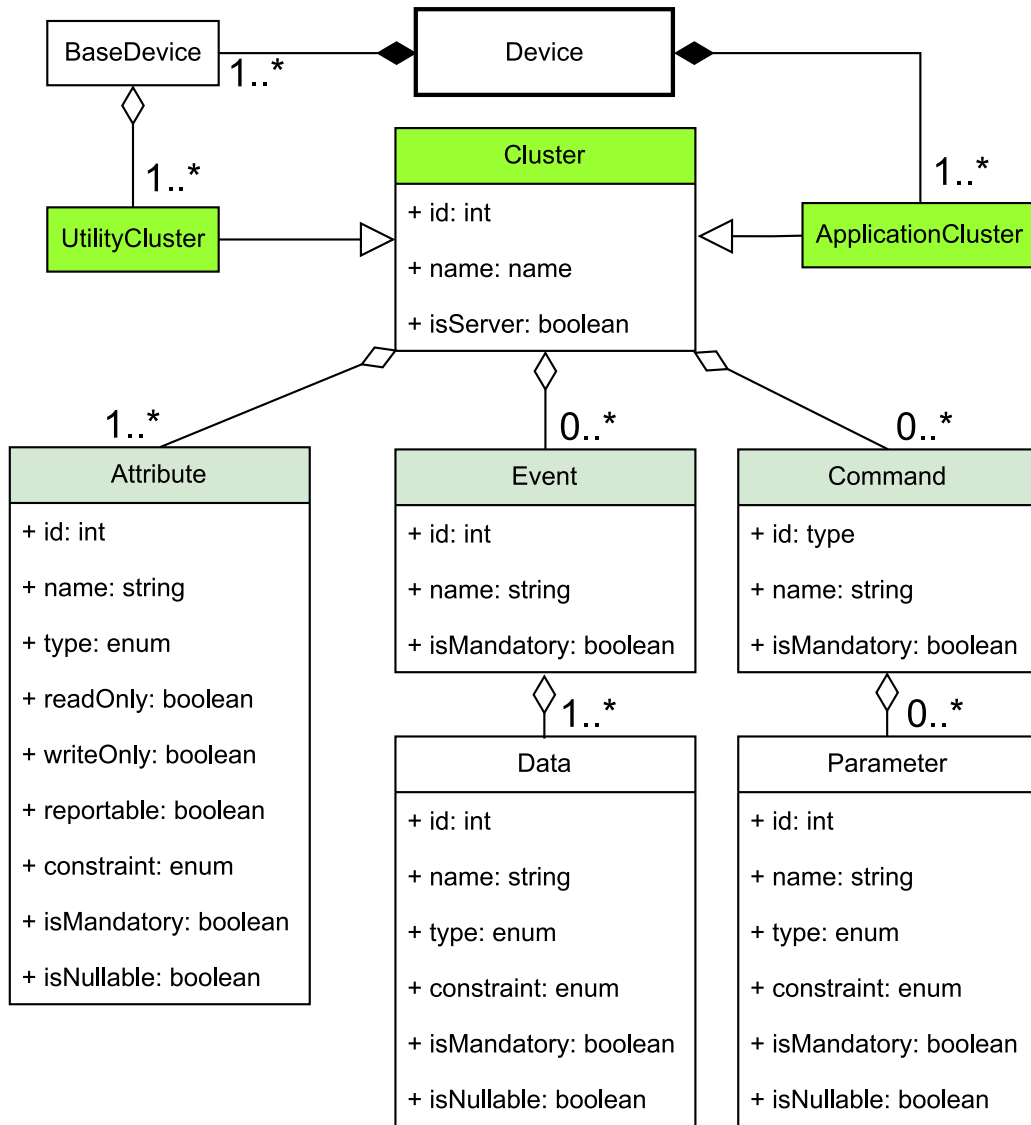


Fig. 8. UML class diagram used to formalize the Matter specification as object-oriented programming (OOP) classes. This diagram enables the Matter data model specification to be systematically processed as OOP objects, facilitating automated model generation and integration.

devices and automatically notifies subscribed applications or services. For writable attributes or commands in Matter, they are mapped to commands in FIWARE, ensuring compatibility and proper handling of device control operations.

5.3. NGSI-LD data model for matter

In NGSI-LD, the first major difference lies in the representation of attribute types. Whereas NGSI-V2 uses primitive types such as Text, Boolean, and Number, NGSI-LD represents these as Property, which is conceptually equivalent to `owl:DatatypeProperty` in the Semantic Web. A second important distinction is the introduction of the Relationship type in NGSI-LD, equivalent to `owl:ObjectProperty`, which allows the definition of links between entities. This feature is particularly valuable in the context of Matter, as it enables the separation of profile-related metadata (e.g., manufacturer information, software/hardware version) from the core functional properties of the entity. Such separation is not natively supported in NGSI-V2, highlighting the advantage of NGSI-LD for semantically enriched data modeling.

NGSI-LD is gaining increasing attention due to its support for semantics and Linked Open Data, which enable more expressive and

interoperable data models. However, to fully realize these advantages, every term in an NGSI-LD data model, including attribute names, must be semantically defined in a context file (i.e., `context.jsonld`). At present, no official context definition exists for the Matter specification. While it is feasible to create an ontology for Matter to provide such semantic context, it remains unclear whether Matter will be an independent ontology or align with existing ontologies such as SAREF [38] or SOSA [39]. Given this uncertainty, and despite our ability to generate NGSI-LD representations, the lack of a supporting ontology renders them semantically incomplete. Therefore, for the purposes of this work, we primarily utilize NGSI-V2 in our experiments.

5.4. Summary

We propose an automated solution for converting Matter device specifications into FIWARE-compliant smart data models, enabling the representation of Matter device information as resources within the FIWARE platform. This integration ensures that Matter device data adheres to the FAIR principles, Findable, Accessible, Interoperable, and Reusable [40], through the abstraction layer provided by FIWARE. The generated data models are essential for enabling the seamless integration of Matter devices into FIWARE-based systems.

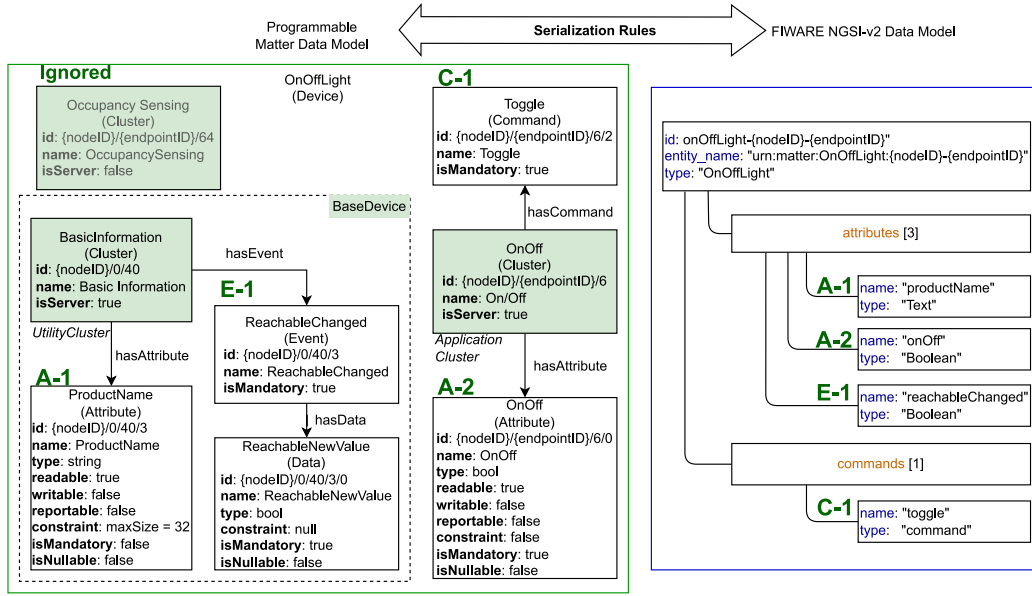


Fig. 9. Example of Mapping the Matter Data Model to FIWARE NGSI-v2: Server clusters and their attributes are mapped to corresponding attributes, events are mapped to attributes, and commands are mapped to commands.

Input : Formalized Matter Device Data Model
Output: NGSI-V2 or NGSI-LD Data Models

```

1 Init1: attributeArray, commandArray = [empty]
2 Init2: isApplicationDeviceType = False
3 for cluster in an endpoint do
    #Select server clusters only
4     if isServer == True then
5         if isApplicationCluster == True then
6             isApplicationDeviceType = True
7         end
8         for attribute in cluster.Attributes do
9             if readable == True OR reportable == True then
10                attributeArray.append(attribute)
11            end
12            if writable == True then
13                commandArray.append(attribute)
14            end
15        end
16        for command in cluster.Commands do
17            commandArray.append(command)
18        end
19        for event in cluster.Events do
20            attributeArray.append(event)
21        end
22    end
23 end
24 #Create FIWARE Entity with data
25 if (attributeArray OR commandArray) != empty then
26     if isApplicationDeviceType == True then
27         1: Init FIWARE Entity with type = Matter Device type
28         2: Append attributeArray to Entity
29         3: Append commandArray to Entity
30         4: Return Entity as JSON schema
31     end
32 end

```

Algorithm 1: Serialization Rules to Generate NGSI-V2 and NGSI-LD Data Models

Our approach has been rigorously tested with Matter specification versions 1.3 (to ensure backward compatibility), 1.4 (the latest stable release), and 1.4.1 (a pre-release version), achieving a 100% success rate in all cases. Specifically, for version 1.4, we successfully converted 72 different Matter application device types into 72 NGSI-V2 entities and 73 NGSI-LD entities, where one additional entity represents device profile information. Although NGSI-LD offers enhanced semantic capabilities, its full potential can only be realized with a corresponding ontology for the Matter specification. Since no such ontology currently exists, this work primarily focuses on NGSI-V2.

Throughout the development process, we identified and reported several typographical and structural errors in the XML representation of the Matter specification, contributing to the refinement of the machine-readable specification. Furthermore, we developed a UML model to formalize the Matter specification, which serves as the foundation for automated data model generation. This formalization not only supports FIWARE integration but also facilitates interoperability with other platforms by providing a structured, extensible representation of Matter device capabilities.

At present, we are in the process of contributing this data model to the official FIWARE data model repository. The details of the proposed data model are presented as follows:

- Domain: dataModel.SmartHome
- Subject: Matter
- Directory: Directory for each device (72 device types)
- Deliverable files: schema.json (and other auto-generated formats), Key-Values for NGSI-v2, Normalized for NGSI-V2, and other auto-generated formats.
- Under discussion: NGSI-LD related files.

6. FIWARE IoT agent for matter

The high-level architecture of the FIWARE IoT Agent for Matter (hereafter referred to as FIMAT) is illustrated in Fig. 10. This component functions as an intermediary between the Matter protocol and the FIWARE ecosystem, implementing the essential mechanisms required for their interoperability. The primary function of FIMAT is to manage Matter devices within smart home environments and ensure their

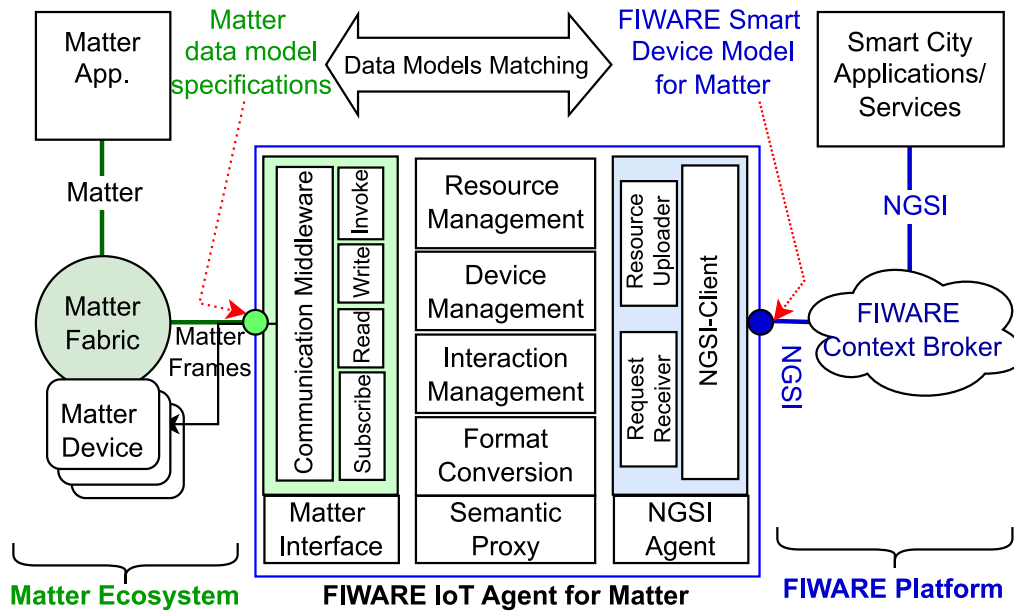


Fig. 10. Detailed design of the FIWARE IoT agent for matter: Managing matter resources (left) and converting them into NGSI resources for FIWARE-compliant services (right)

Findability, Accessibility, Interoperability, and Reusability (FAIR) for external services through standardized FIWARE APIs. The following sections outline the core components of FIMAT.

- **Matter Interface** is a software component that complies with the Matter specification and integrates communication middleware to enable interactions with Matter-certified devices. These interactions include reading data, writing data, subscribing to device updates, and invoking device actions. This interaction model establishes fundamental primitives that serve as a foundation for other modules to efficiently manage and control Matter devices.
- **Semantic Proxy** functions as an intermediary between Matter and FIWARE, facilitating the translation of data and semantics between these two systems.
 - **Format Conversion** is responsible for translating both data and control messages between the two protocols. This module utilizes the proposed FIWARE Smart Data model to interpret and process FIWARE resources, while also leveraging the programmable Matter data model to decode and process the Matter frames, as described in the previous section.
 - **Interaction Management** module utilizes the primitive actions defined in the Matter specification to implement algorithms that facilitate effective interactions with Matter devices. For example, if a device's attribute supports a notification mechanism, the module can leverage the subscription mechanism to receive updated data whenever the attribute changes, rather than repeatedly polling the device to retrieve the latest information.
 - **Device Management** module is tasked with implementing mechanisms for overseeing Matter devices within the system. This includes functions such as commissioning new devices, removing unresponsive or potentially faulty devices from the managed fabric, and performing routine maintenance tasks. These tasks involve conducting health checks, monitoring battery levels, and gathering diagnostic information to ensure proper device operation. The module also provides alerts in the event that a device is approaching failure, allowing for timely intervention.

- **Resource Management** module is responsible for maintaining a digital representation of physical devices, ensuring that the latest state of these devices is accurately synchronized within the FIWARE Context Broker. In the case of a device flagged by the previous module as being at risk of failure, the module temporarily removes the device from the broker. This prevents other services from interacting with devices that are either failing or nearing failure.

- **NGSI Agent** implements standard FIWARE IoT Agent functionality to facilitate seamless interconnection with FIWARE Context Brokers. For northbound traffic, which involves transmitting measurements from Matter devices to FIWARE, the agent may register a new entity corresponding to an added Matter device in the home or update the attributes of an existing entity to reflect the latest state of the physical device using the NGSI-Client. Conversely, for southbound traffic, which involves commands originating from the Context Broker, the agent receives these commands, which are then translated into Matter commands by the Semantic Proxy module and executed by the Matter Controller module.

The FIMAT is designed with an event-driven architecture, which handles two types of events: events from Matter devices and events from the FIWARE Context Broker. A non-exhaustive list of events supported by FIMAT is summarized in [Table 5](#).

In summary, FIMAT adheres to standard methodologies for designing and implementing an IoT Agent within the FIWARE ecosystem. While each IoT Agent is inherently distinct due to differences in protocol interpretation, significant commonalities exist across implementations. These include:

- An Interface for Device Updates (Matter Interface): A designated location to receive updates from devices.
- Standardized Interface for Context Data Updates (NGSI Agent): A dedicated endpoint to process updates to contextual data.
- Device Registry and Attribute Mapping (Semantic Proxy): Maintaining a catalog of devices and mapping context data attributes to the corresponding device-specific syntax (Matter Frame).

The following sections outline the two procedures for initializing the FIMAT and the steps involved when an application controls Matter devices via FIWARE Context Broker.

Table 5

A non-exhaustive list of events supported by the FIWARE IoT Agent for Matter.

Event	Description	Actions
NodeAdded	A physical matter device has been commissioned to the fabric successfully	Register entities to the FIWARE Context Broker: - entities if the node contains several application endpoints - a single entity if the node contains only one
NodeRemoved	A physical matter device has been removed from the fabric	Unregister related entities from the FIWARE Context Broker
NodeEvent	A node has become unavailable or has become available again	Unregister or register the corresponding entities from the broker as needed
EndpointAdded	A new endpoint has been added to the logical node	Register entity if the endpoint is application endpoint
EndpointRemoved	An endpoint has been removed to the logical node	Un-register corresponding entity
AttributeUpdated	An attribute of an endpoint of the node has been changed	Update corresponding attribute of the corresponding entity
ContextEntityUpdated	An entity's context data has been updated	Control the corresponding Matter device accordingly
ContextAttributeUpdated	An entity's attribute context data has been updated	Control the corresponding Matter device accordingly
CommandInvoked	A command has been sent from the context broker	Control the corresponding Matter device accordingly

6.1. Southbound traffic: Register matter endpoint as FIWARE entity

The algorithm responsible for registering the FIMAT itself and its managed devices is summarized in Algorithm 2. Initially, the agent registers itself with a pre-configured FIWARE Broker using the standard procedure. At this point, the agent acts as an external context provider, serving as the entry point to access Matter device resources.

Input : FIWARE Context Broker Configuration

```

1 Initialize the IoT Agent
2 if success then
3   for commissionableDevice in MatterFabric do
4     Execute commissioning process
5     if success then
6       Get device type and supported clusters and initial
         values of every endpoint from the node
7       if endpoint is ApplicationEndpoint then
8         Convert to FIWARE Entity
9         Register entity to the broker
10        Subscribe to the entity's context in the broker
11        Init a thread to monitor physical device
12        if latestValue != value in memory then
13          Update entity's context in the broker
14        end
15      end
16    end
17  end
18 end

```

Algorithm 2: Initialization Procedure of the FIMAT

In the subsequent phase, the resources of Matter devices will be translated into NGSI resources and registered as context data within the FIWARE Context Broker. To facilitate registration by the agent, the Matter device must be in a commissionable state, enabling the agent to initiate the commissioning process. This procedure involves verifying the device certificate and performing the necessary steps to establish a secure connection. Upon successful commissioning, the device's information is registered as the corresponding entity. During this process, the device type, supported clusters, and initial attribute values are retrieved and mapped to the corresponding NGSI-v2 model.

To ensure that the context data in the FIWARE Context Broker remains up-to-date with the latest information from physical devices, a dedicated thread is assigned to each commissioned Matter device. These threads continuously monitor device attributes by leveraging the subscription interaction mechanism of the Matter protocol. The FIMAT acts as an initiator, utilizing Subscription Interaction to automatically receive periodic Report Data transactions from the device. To maximize efficiency, we employ the **wildcard** subscription

mechanism, ensuring that any attribute change event is immediately reported. The most recent data is then updated in the corresponding entity within the FIWARE Context Broker, maintaining real-time synchronization with physical devices.

6.2. Northbound traffic: Processing commands from context broker

In general, when an application or service interacts with IoT devices through the FIWARE Context Broker, it does not require detailed knowledge of the underlying connectivity mechanisms or communication protocols of the devices. Instead, IoT devices are represented as contextual data managed by the broker. Typically, interactions involve issuing commands (as outlined in Tables 1 and 2) to either obtain the most recent device data, through a retrieval request or a subscription mechanism, or to control the device by updating attributes associated with its physical state. This section illustrates both data retrieval and control processes through a use case involving a Matter lightbulb.

Fig. 11 presents a simplified sequence diagram illustrating how FIMAT synchronizes the state of a physical device with its digital representation within FIWARE. Without this caching mechanism, an application or service would need to wait for the Context Broker to explicitly request data from the physical device before providing a response. This approach negatively impacts both system performance and responsiveness. By contrast, the proposed caching mechanism ensures that the most recent state of physical devices is consistently maintained within the Context Broker. As a result, applications can directly retrieve the latest device state from the broker without additional delays. Furthermore, this mechanism, in conjunction with the FIWARE Context Broker's notification mechanism, enables applications to subscribe to relevant contextual data and receive updates automatically whenever changes occur.

Fig. 12 illustrates the sequence of steps involved in controlling a smart lightbulb through an application via the FIWARE Context Broker. The application invokes a command to control a device via its corresponding entity within the broker. For clarity, access control procedures are omitted from this representation. The broker is responsible for managing and delegating the command to the appropriate IoT Agent. Upon receiving the command, formatted in NGSI, the IoT Agent processes it, generates the corresponding Matter control frames, and transmits them to the target device. Once the Matter device receives the command, it executes the requested action and reports the execution result. Subsequently, the IoT Agent updates the Context Broker to reflect the actual device state. If the application employs the notification mechanism, the broker automatically notifies it of the updated context data.

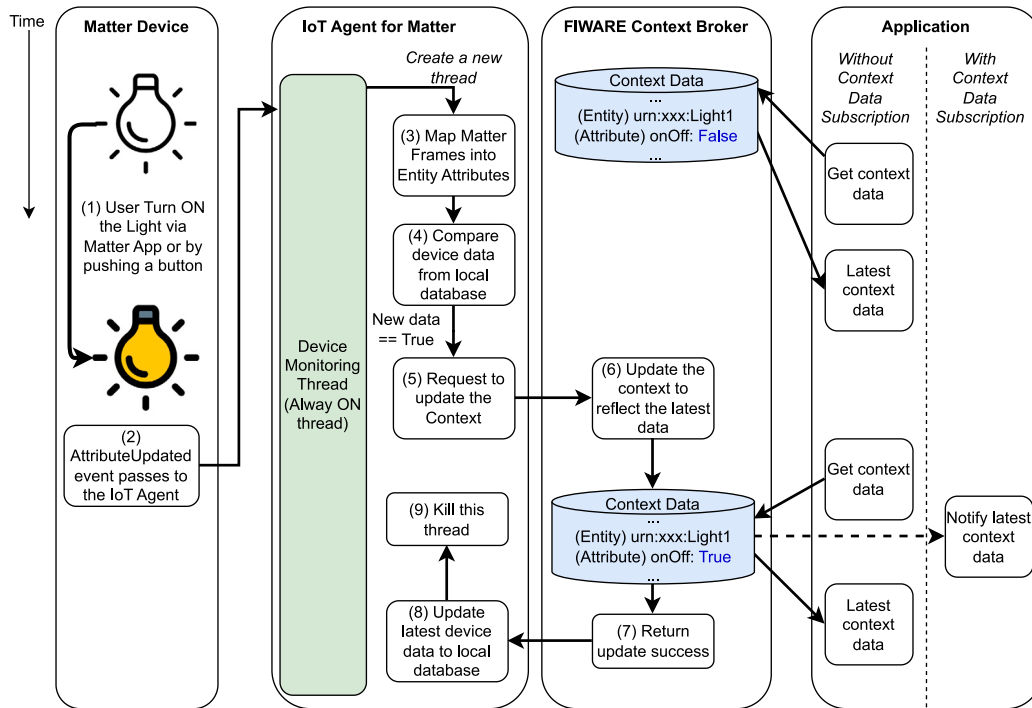


Fig. 11. Simplified Sequence of Application Context Data Retrieval from the Broker: The context data is continuously updated with the latest values of physical devices by the IoT Agent. This ensures that applications interact exclusively with the FIWARE Context Broker without the need for direct communication with the devices themselves.

6.3. Summary

In the described sequence, a caching mechanism is employed to enhance system scalability and performance. Instead of direct request-response interactions with physical devices, applications communicate with them via FIMAT, which maintains a consistent reflection of the devices' latest states. This design avoids the block-and-wait phenomenon and improves overall system efficiency. However, a drawback of this approach is the need for more frequent interactions with Matter devices to obtain up-to-date information, leading to increased network traffic and higher power consumption [41].

Nevertheless, as detailed in the implementation section, this work does not adopt the traditional polling mechanism described in [41]. Instead, it leverages the *Subscribe* interaction, one of Matter's native communication primitives, through which devices proactively report their state changes to FIMAT. This approach significantly reduces network overhead and is particularly well-suited for supporting Long Idle Time devices introduced in Matter specification version 1.4.

The second requirement of the caching mechanism is the implementation of a health-check system to detect unresponsive devices. Devices identified as unresponsive must either be flagged accordingly or removed from FIMAT to prevent applications from interacting with unavailable resources. To address this, we adopt the health-check mechanism previously developed in our work [35], illustrated in Fig. 13.

This mechanism leverages the General Diagnostics cluster, mandated by the Matter specification to log failure-related information, as the primary tool for assessing device health. If a device fails to report its General Diagnostics data within a defined interval, the system attempts to retrieve data from the device's Application clusters as a fallback. If this secondary query also fails, the system initiates a series of retry attempts, up to a predefined threshold. Should the device remain unresponsive beyond the allowable duration, a `NodeEvent: Node has become unavailable` event is triggered. This event prompts FIMAT to unregister the corresponding entity from the broker,

thereby preventing applications from interacting with non-functional devices and ensuring overall system robustness.

Using `tcpdump` to capture traffic between the TP-Link Tapo smart plug and FIMAT, we observed that the device periodically emits a constant UDP status message (161 bytes) containing its current power-consumption reading. This behavior is expected, as Tapo smart plugs support real-time energy monitoring. Even when no load is connected, the plug reports its baseline power consumption, which reflects the energy required to maintain network connectivity. These periodic reports generate a continuous low-rate sensor data stream toward FIMAT. Because the smart plug is powered, this traffic imposes no impact on the device battery life. The network impact is also minimal: each report (and similar for other types of devices) is only twice the size of a standard ICMPv6 Neighbor Solicitation packet (86 bytes). Therefore, the additional load does not meaningfully affect overall network performance.

However, the situation is different for battery-powered devices, which are designed to spend most of their time in sleep mode and only transmit when measurements change (e.g., temperature, humidity, or binary contact sensors). For such devices, unsolicited periodic reporting would significantly increase energy consumption and shorten battery lifetime. Fortunately, environmental sensors typically exhibit low-rate, slowly varying measurements, making a longer reporting interval practical. Therefore, the caching mechanism used in FIMAT remains effective, while adjusting the reporting frequency to a slower rate provides a viable strategy to mitigate the energy cost associated with continuous updates.

7. Implementation and evaluation

7.1. FIWARE IoT agent for matter: Implementation

To validate the feasibility of the proposed data model and the design of FIMAT, we have implemented the conceptual framework as a fully functional software solution. This implementation has been deployed on embedded hardware to assess its overall performance. The detailed software modules of the implementation are outlined below.

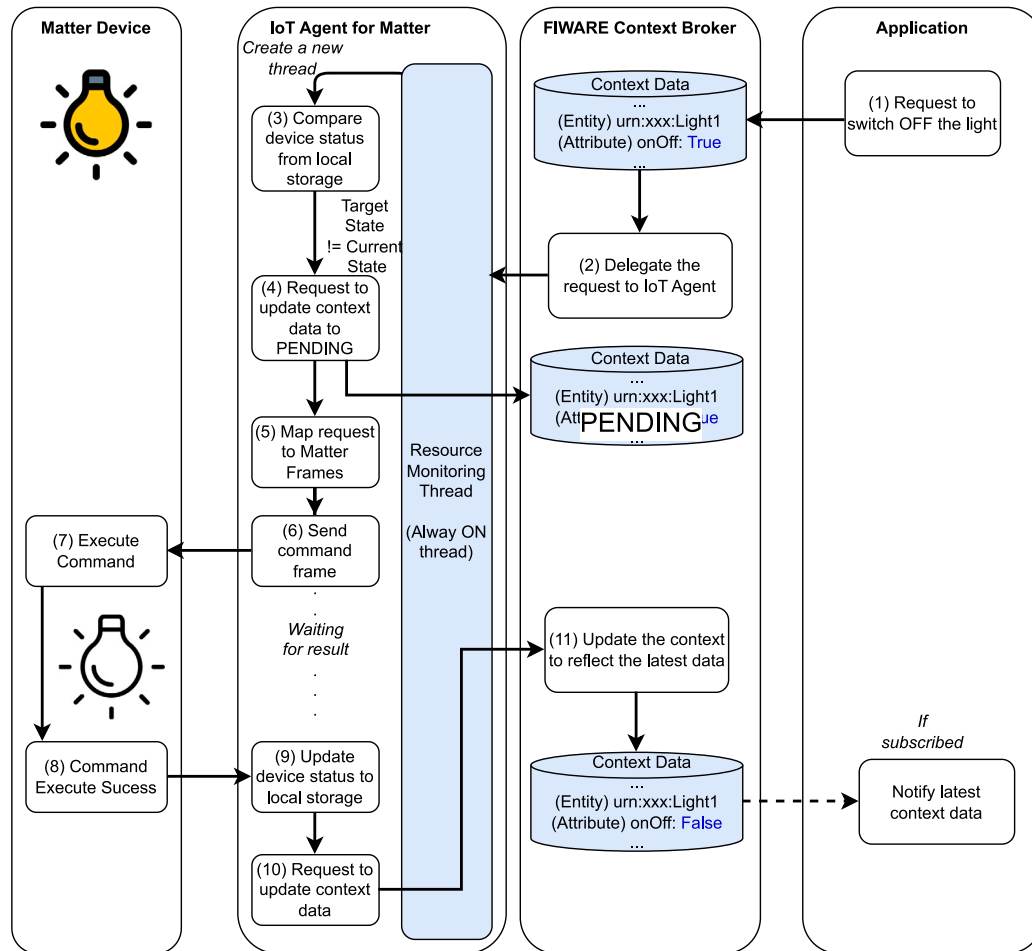


Fig. 12. Simplified sequence illustrating how an application controls a physical device via FIWARE. The application issues a command to the corresponding entity, prompting the Context Broker to delegate the command for device control and state updates. The application interacts solely with the broker and does not directly communicate with the IoT Agent.

Table 6
Open-source matter middleware solutions.

Controller for matter	Language/platform	Key features	Usecase
Open Home Foundation Matter Server	Python/Linux	-GUI based -Cloud platform Independent -Websocket supported	Local controller for Matter, multi-protocol integration
CHIP-Tool	C++/ Linux, MacOSWindows	-CLI based -Official Matter tool -Websocket supported	For development and testing purposes
Matter.js	TypeScript/ Node.js	-Controller as a Module -Support both device of controller	Good for web-based applications

- **Matter Interface:** Table 6 presents several potential candidates for a Matter interface capable of managing and interacting with Matter devices.

For this study, we adopted the solution provided by the Open Home Foundation, the `python-matter-server`⁶ version 7.0.1, which follows a universal approach, enabling seamless integration with the FIWARE platform. This solution serves as the core component for providing Matter support to Home Assistant, a

widely used smart home platform. It facilitates communication via a WebSocket connection, allowing commands to be sent as text messages for device control and enabling the reception of data from Matter devices in JSON format.

- **Semantic Proxy** primarily implements mechanisms for monitoring events occurring in Matter devices, such as the addition or removal of a device from the Matter fabric, status changes, or unresponsiveness. Additionally, it translates Matter-related messages exchanged via WebSocket between the Matter interfaces and NGSI resources, ensuring compatibility with NGSI agents. The semantic proxy employs a rule-based approach for message

⁶ <https://github.com/home-assistant-libs/python-matter-server>

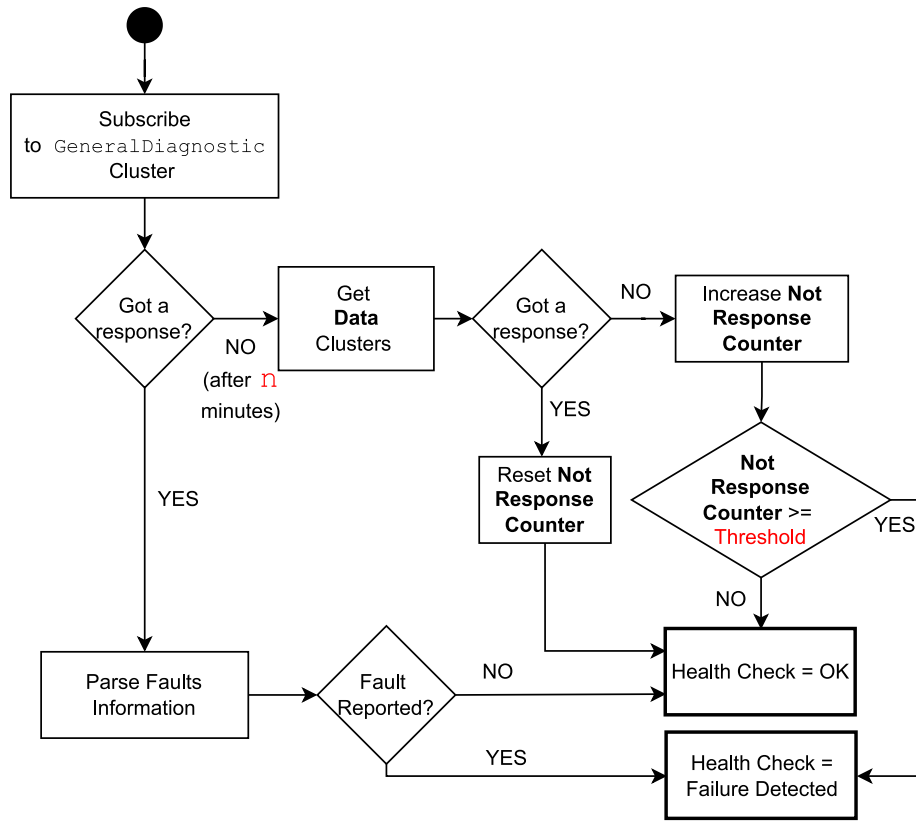


Fig. 13. Simplified flowchart of the health-check process.

translation and utilizes event-driven notifications to activate the corresponding translation rules.

- **NGSI Agent:** Table 3 presents several potential NGSI agents for integrating Matter with FIWARE. Although the data received by this agent is in JSON format, the IoT Agent for JSON was not selected, as it functions as an HTTP/MQTT/AMQP endpoint, introducing additional latency. Instead, we utilize the core library, `iotagent-node-lib`⁷ version 4.7.0, to minimize communication delays and enhance efficiency.

We utilize Home Assistant OS due to its compatibility with the previously introduced software stack. The `python-matter-server` is installed as a Home Assistant add-on, providing Matter support. Additionally, we developed a custom add-on using Node.js, which integrates both the `iotagent-node-lib` and the Semantic Proxy. These two add-ons communicate via a local WebSocket connection.

7.2. FIWARE IoT agent for matter: Evaluation with matter devices

To evaluate the feasibility and effectiveness of the proposed solution, we conducted tests using a variety of devices, including experimental, commercial, and emulator Matter devices. This comprehensive approach allowed us to assess the performance and compatibility of our implementation across different types of devices and environments, providing valuable insights into its potential real-world applicability and robustness.

7.2.1. Experiment configuration

The overall experimental setup is shown in Fig. 14. In this configuration, the Matter devices and the FIMAT system connect to a smart-home network via Wi-Fi (2.4 GHz) provided by a wireless access

point (Buffalo WRS3200AX4S, Wi-Fi 6). The access point uplinks to a Gigabit Ethernet switch (Netgear GS108) through a 1000 Mbps port. The FIMAT system communicates with the Orion Context Broker (version 4.1.0), which is hosted on a separate PC located in a different subnet. This communication occurs over a 1000 Mbps wired Ethernet link through the same Netgear GS108 switch. To interact with the Matter devices, Postman is used as the client application to issue HTTP requests to the context broker, enabling GET and SET operations on device states. The testing applications run on another PC, also connected to the GS108 switch via a 1000 Mbps wired connection. For comparative evaluation, an MQTT broker (Eclipse Mosquitto, version 2.0.21) was deployed on identical hardware and using the same network topology as the Orion Context Broker. This configuration enables a direct performance comparison between the FIWARE-based setup and a lightweight, bare-metal MQTT solution. The packet loss of the setup using the `ping` command is 0% (100 packets transmitted, 100 packets received) for all connection paths.

The list of devices, along with their supported clusters and the corresponding FIWARE entities and attributes in the FIWARE Smart Data Models for Matter, is summarized in Table 7.

7.2.2. Experiment objectives

The experiment is designed to achieve two key objectives:

- Validation of the FIWARE Smart Data Models for Matter (Experiment 1).
- Evaluation of the operational performance of the proposed IoT Agent, including:
 - Experiment 2: Verification of plug-and-play registration of Matter devices as FIWARE Entities.
 - Experiment 3: Assessment of state synchronization between physical Matter devices and context data in the FIWARE broker.

⁷ <https://github.com/telefonicaid/iotagent-node-lib>

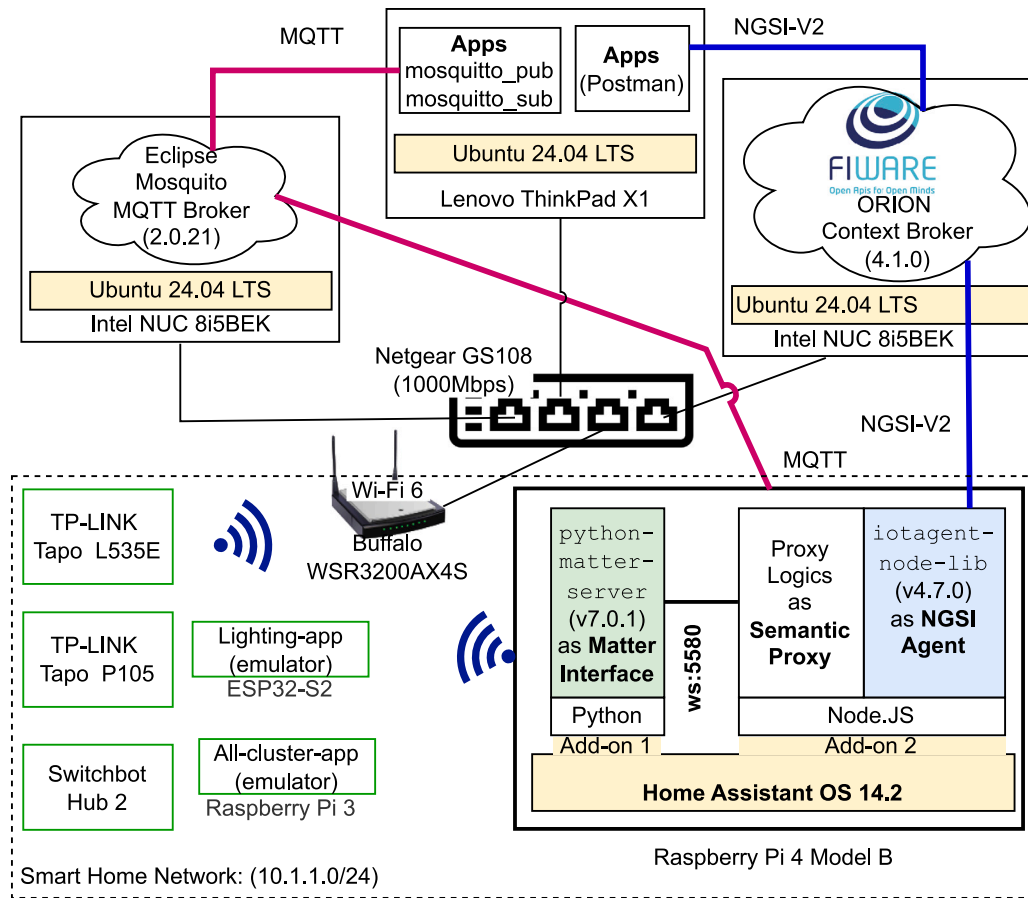


Fig. 14. The experimental setup illustrating the network configuration and the logical connection paths.

- Experiment 4: Evaluation of the IoT Agent's capability to handle requests forwarded by the broker.
- Experiment 5: Load test with JMeter to confirm the operation of the IoT Agent's capability to handle concurrent requests.

All experiments (except Experiment 1) were repeated 20 times. During each run, FIMAT recorded every execution step in its internal log. Using these logs, we computed the round-trip time (RTT) for each trial. FIMAT was deployed on a Raspberry Pi 4 (8 GB RAM). At idle, the device exhibited 1%–2% CPU usage and approximately 0.8 GB of memory usage by the operating system. The memory usage of the deployed FIMAT software stack is approximately 100 MB.

7.3. Experiment result

7.3.1. Experiment 1: Verify the proposed data model

A well-defined data model is essential for representing physical Matter devices as FIWARE context data in an interoperable manner with third-party applications and services. In this experiment, we evaluated whether the proposed data model sufficiently captures the Matter data model specifications.

As summarized in Table 7, we tested FIMAT with both certified and emulated Matter devices. For certified devices, which strictly conform to the Matter data model specifications, we verified that our proposed data model successfully maps 100% of the physical device attributes into FIWARE context data.

Additionally, we utilized the all-clusters-app, a sample application provided as part of the Matter SDK, to emulate a virtual On/Off Light device supporting all available clusters specified in the Matter

specifications (for testing purposes). Since no physical device implements this complete set of clusters, and the Matter specification for an On/Off Light device only includes the On/Off Cluster, our data model mapped only the attributes of this cluster to the corresponding OnOffLight entity, while the remaining clusters were excluded. These results confirm the validity of the proposed data model in accurately representing the Matter specification while demonstrating its ability to accommodate variations in device implementations by filtering out undefined concepts.

7.3.2. Experiment 2: Plug-and-play entity registration

The fundamental sequence of the FIMAT when processing an event indicating that a Tapo Light P105 has been added to the Matter fabric is summarized in Fig. 15. The commissioning process is omitted, as it occurs only once and follows standardized procedures defined by the Matter specification.

First, the FIMAT retrieves essential information from the devices, including endpoints (e.g., endpoint 0, endpoint 1), supported clusters, and the current values of each attribute. For the lightbulb used in the experiment, this process took a total of 146.7 ms (min/max/stddev values are 127.1/152.5/7.2 ms).

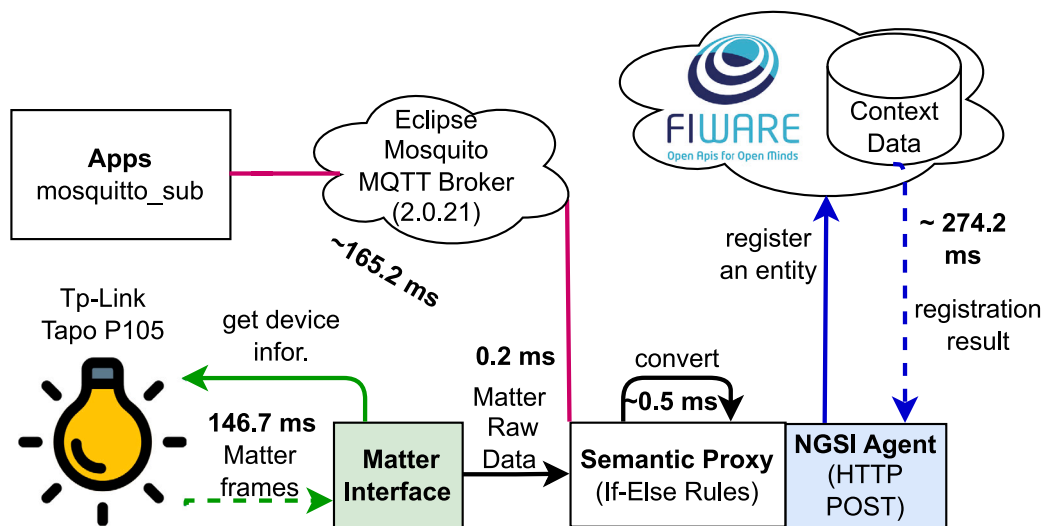
Next, the retrieved Matter raw data is processed by the Semantic Proxy. Since a local WebSocket connection is utilized, the transmission time is minimal, measuring 0.2 ms (min/max/stddev values are 0.172/0.221/0.016 ms). Using the data model introduced in the previous section, the FIMAT converts the raw data into a JSON payload following the NGSI-v2 format, which requires 0.5 ms (min/max/stddev values are 0.172/0.221/0.016 ms) for transformation.

The formatted payload is then processed by the NGSI agent, which sends an HTTP POST request to register the device as context data in

Table 7

List of Matter devices and their corresponding Entities in the experiment.

Physical device	Matter devicetype	Clusters	FIWARE entity	Attribute
TP-LINK Tapo L535E	On/Off Plugin Unit	On/Off	OnOffPluginUnit	onOff
TP-LINK Tapo P105	Extended Color Light	On/Off	ExtendedColorLight	onOff globalSceneControlOnTime offWaitTimestartUpOnOff
		LevelControl		currentLevel remainingTime onLevel onTransitionTime offTransitionTime
		ColorControl		currentHue currentSaturation remainingTime currentX currentY colorTemperatureMireds colorMode primary2X primary2Y enhancedColorMode colorCapabilities colorTempPhysicalMinMireds colorTempPhysicalMaxMireds startUpColorTemperatureMireds
Switchbot Hub 2	Temperature Sensor	Temperature Measurement	TemperatureSensor	measuredValue minMeasuredValue maxMeasuredValue
		Relative Humidity Measurement	HumiditySensor	measuredValue minMeasuredValue maxMeasuredValue
Emulator Lighting-App	Dimmable Light	On/Off	DimmableLight	onOff globalSceneControlOnTime offWaitTimestartUpOnOff
		LevelControl		currentLevel remainingTime onLevel onTransitionTime offTransitionTime
Emulator All-cluster-App	On/Off Light	On/Off	OnOffLight	onOff globalSceneControlOnTime offWaitTimestartUpOnOff
		All other Matter clusters		Ignored

**Fig. 15.** Average Time Required for Registering a Matter Device as a FIWARE entity using the proposed IoT Agent.

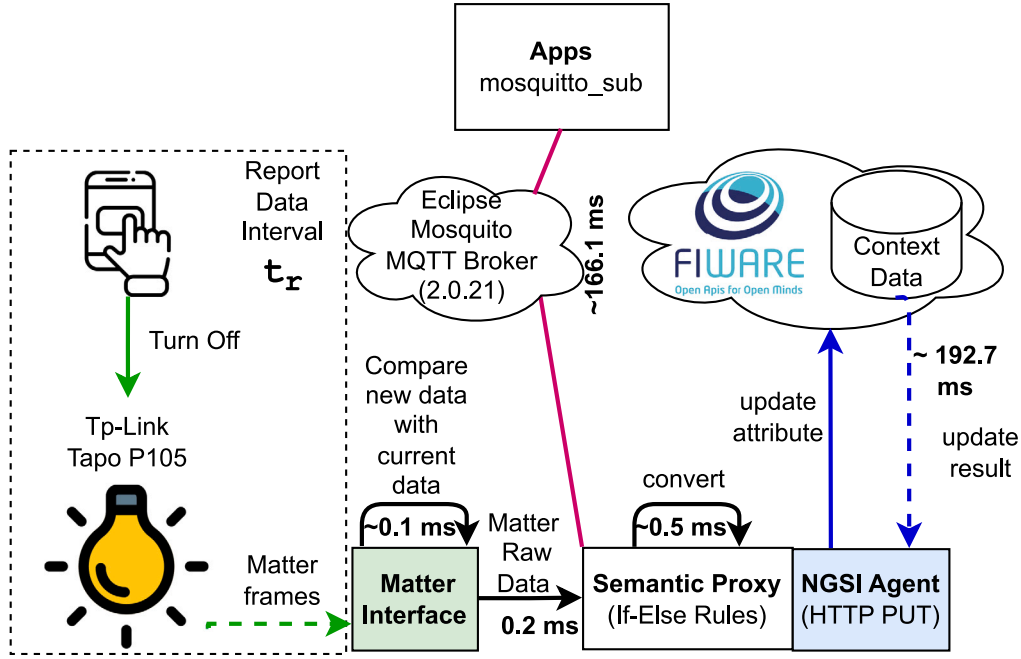


Fig. 16. Average time required to update physical device state to the FIWARE Context Broker.

FIWARE. This step, including processing and network communication, took **274.2 ms** (min/max/stddev values are 216.7/291.0/20.2 ms) for transformation.

In total, the entire process required approximately **422.3 ms** (min/max/stddev values are 360.9/470.2/24.3 ms) to register the lightbulb as an entity in FIWARE. Although the registration time may vary depending on the device, the average processing time for all tested devices in this experiment remained below **500 ms**.

To compare the performance of direct MQTT communication versus FIWARE-based context updates, we conducted an additional experiment where raw Matter data was published directly to an MQTT broker, and an MQTT subscriber was used to receive the data. In this setup, the end-to-end latency for data delivery to the subscriber was measured at **165.2 ms** (min/max/stddev values are 147.3/192.0/10.9 ms). In contrast, when the same data was processed and converted into context data within the FIWARE broker, the total time was **274.2 ms** (min/max/stddev values are 243.6/291.7/13.6 ms). This indicates an overhead of approximately **109 ms** introduced by the context data processing and updating database operations in Orion Context Broker.

The add-on CPU usage when this experiment was conducted was negligible (always 1%), and the additional memory usage was increased by 5 MB (0.1%)

7.3.3. Experiment 3: Synchronize matter device state to FIWARE broker

As detailed in Section 6.2, we employ the Subscribe Interaction mechanism to request Matter devices to report their current snapshot data. The key parameters of a subscription request, along with the node identifier, include:

- Min interval floor (minimum interval between Data Reports): $t_{r,min}$.
- Max interval ceiling (maximum interval between Data Reports): $t_{r,max}$.
- Request for attribute and/or event paths to be reported.

In the experiment presented in Fig. 16, we set $t_{r,min}$ to 0 s (as recommended by the specification to better handle devices with long idle periods) and $t_{r,max}$ to 10 s, utilizing the wildcard `0xFFFFF` to retrieve all available data from the light. This polling rate is adjustable

and presents a trade-off: reducing $t_{r,min}$ and $t_{r,max}$ enhances system responsiveness in real time but may lead to increased battery consumption and impact overall system performance. The 10-second parameter can be adjusted to suit various configurations; in our experiment, it was set according to the configuration specified in [42].

The time required for a complete update cycle of the lightbulb's state is illustrated in Fig. 16. Once the Matter frames arrive at the Matter interface, the system takes less than 0.1 ms to compare the newly received data snapshot with the previously stored snapshot in memory. If a change is detected (e.g., a state transition from On to Off), only the updated data is forwarded to the Semantic Proxy, where it is converted into a JSON payload for an attribute update request in the FIWARE Context Broker.

In our experiment, the average time required for this process varied depending on the device's performance. The recorded average update time was **193.5 ms** + t_r (min/max/stddev values are 172.7/204.1/9.8 ms). In our experiment, the response time on the device side, denoted as t_r , is negligible; the light turns on almost instantaneously after the button is pressed. Since t_r is both difficult to measure accurately and highly dependent on the internal performance of the device, it can be reasonably excluded from the overall latency analysis.

In comparison with the bare-metal MQTT broker, where the latest state of the light device was received by the subscriber in **166.1 ms** (min/max/stddev values are 148.4/197.1/10.6 ms), the Orion Context Broker required **192.7 ms** (min/max/stddev values are 171.0/206.3/8.4 ms) for the same operation. This indicates that approximately **26.6 ms** was incurred as additional overhead by the Orion Context Broker for updating the context database.

In the worst-case scenario, when $t_r = t_{r,max}$, there is a 10,193.5 ms delay in reflecting the light's state in the context data. However, in the best-case scenario, if the light is turned on just before it sends the Report Data, the update process can be completed in less than 200 ms. Additionally, within the $t_{r,min}-t_{r,max}$ time window, if the light is toggled twice, these events are ignored by FIMAT, as there is no net change in the final state of the device.

The add-on CPU usage when this experiment was conducted was negligible (always 1%), and the additional memory usage was increased by 5 MB (0.1%)

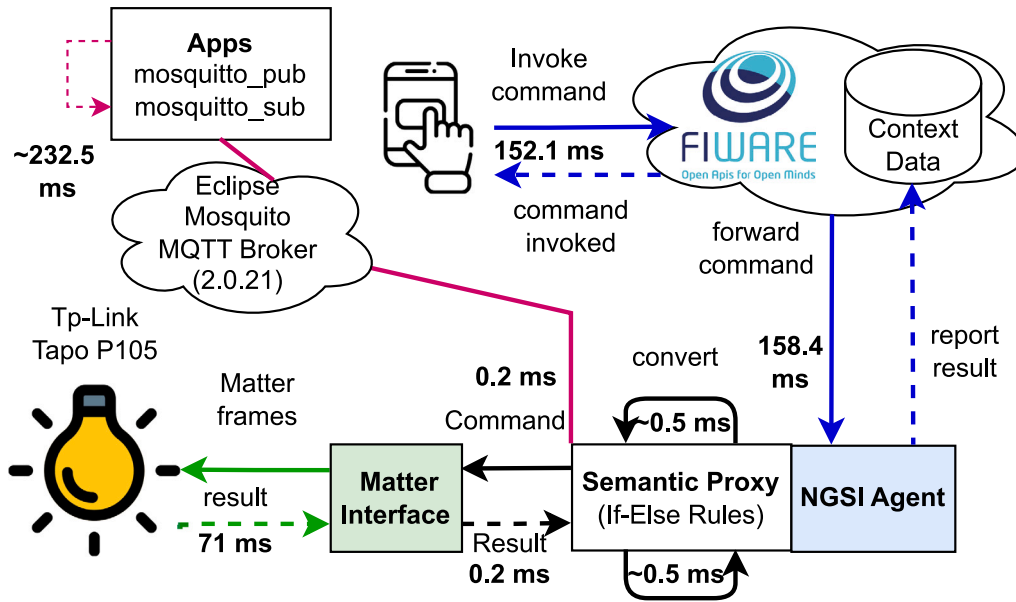


Fig. 17. Average time required to update physical device status in response to requests sent by an application via the context broker.

7.3.4. Experiment 4: Processing commands from context broker

In this experiment, we utilize Postman as an application to send commands for turning the Tapo Light on and off via the FIWARE Context Broker. Notably, controlling the light does not require any knowledge of its underlying protocol or how it is connected to the broker.

As described in Section 6.2, the FIMAT listens for commands sent by the broker via a dedicated HTTP endpoint that is configured to handle notifications. When the command is triggered by the Postman client, the broker takes 152.1 ms (min/max/stddev values are 131.9/180.3/13.5 ms) to confirm that the command has been successfully invoked (this confirmation does not indicate that the light has already switched on or off). The command is then processed by FIMAT according to the workflow outlined in Section 6.2. As summarized in Fig. 17, the entire process, from receiving the command to executing the action on the light and updating the context data at the broker, takes a total of 230.8 ms (min/max/stddev values are 201.2/247.6/11.9 ms).

For comparison, the device control command was issued by publishing a message to the Matter Interface via the MQTT broker, with the response received through an MQTT subscriber. The total time required for this end-to-end operation was 232.5 ms (min/max/stddev values are 192.9/244.6/13.2 ms). In contrast, performing the same operation through the Orion Context Broker introduced an additional 150.4 ms of latency. This overhead is attributed to the processing handled by the IoT Agent and is consistent with the interaction sequence shown in Fig. 12, where the context broker updates the context database twice, once when the command is issued and again when the updated device state is reported.

The add-on CPU usage when this experiment was conducted was negligible (always 1%), and the additional memory usage was increased by 5 MB (0.1%).

7.3.5. Experiment 5: Load test with JMeter

This experiment evaluates the ability of the proposed system to handle concurrent requests at the FIWARE interface layer. We used Apache JMeter [43] on the test machine shown in Fig. 14 to generate controlled HTTP load toward the Orion Context Broker. The broker was deployed on a mini PC equipped with an Intel Core i5 processor and 8 GB RAM, without any load balancer or distributed Orion configuration. Since a standalone Orion instance has a default limit of 1020 concurrent

connections, we executed the load test using 10, 100, and 1000 requests per second, corresponding to typical and near-limit stress conditions. Each configuration was executed 20 times, and the aggregated results are presented in Table 8. Because the SET operation in our architecture intentionally uses a *block-and-wait* mechanism to prevent malformed or malicious control sequences, the stress test focuses exclusively on GET-related operations, which represent the primary data-retrieval pattern. Moreover, FIMAT is designed around a **caching** model, where the broker maintains the latest device states. Therefore, high-rate GET traffic exercises the Orion broker but does not affect the responsiveness of FIMAT or the underlying Matter devices.

On the tested minimal hardware, the Orion broker begins to degrade at around 1000 requests/second, which approaches its default connection limit. In real smart-city deployments, however, the broker is typically deployed using horizontal scaling, distributed brokers, and load-balancing components. Under such architectures, throughput is significantly higher, and the bottlenecks observed in this experiment would not arise. Thus, the results of this micro-benchmark should be interpreted as a reference baseline on minimal hardware, while also highlighting that the caching-based design of FIMAT ensures that the system remains scalable and operational even under high query loads in real-world deployments.

7.4. Comparative analysis

Since this study is the first to propose a FIWARE IoT agent for Matter, we provide a comparative analysis of our approach alongside related works in Table 9. This research focuses on bridging smart home and smart city environments, similar to the work presented in [29]. However, while [29] centers on ECHONET Lite, a protocol primarily adopted in Japan, our study emphasizes Matter, an emerging global standard.

Additionally, the work in [35] implements a controller that integrates Matter with oneM2M, a general-purpose cloud platform, effectively mitigating vendor lock-in at the platform level. However, their approach relies on chip-tool as the Matter interface, which lacks scalability. Moreover, their interaction model is based on a request-response paradigm, which is inefficient, particularly for battery-powered devices.

Similarly, [44] adopts the same Pub/Sub communication pattern as our study and extends the FIWARE MQTT IoT Agent, making it

Table 8
System Load Evaluation (Number of concurrent request $N = \{10, 100, 1000\}$)

Number of concurrent request per second	Response time (ms)			Error rate (%)		
	Min	Max	Mean	Min	Max	Mean
10	37	45	39	0.01	0.01	0.01
100	36	47	40	0.01	0.01	0.01
800	49	72	59	0.14	0.25	0.19
1000	53	128	76	0.30	0.43	0.37

Table 9
Comparative analysis of this work and related studies.

	IoT agent for ECHONET	IoT agent for MQTT	IoT agent for matter	Matter controller oneM2M
Domain	Smart home x Smart city	Smart agriculture x Smart city	Smart home x Smart city	Smart home x M2M platform
Target Integration	ECHONET Lite and FIWARE	MQTT-based devices and FIWARE	Matter and FIWARE	Matter and oneM2M platform
ITU-T Y.2070 Layer	Gateway	Gateway	Gateway	Gateway
Data model proposal?	NO	NO	YES	YES
Device Error Handling?	NO	NO	YES	YES
Device Interaction Model	Request/Response	Pub/Sub	Pub/Sub	Request/Response

comparable to our approach. Nevertheless, it does not introduce a standardized data model, thereby limiting reusability across applications and services. Furthermore, it lacks a mechanism for handling device errors, which is a critical limitation given the susceptibility of IoT devices to malfunctions. Such issues, if unaddressed, could lead to significant disruptions within the FIWARE ecosystem, as discussed in the previous section.

Conducting a direct performance comparison is challenging, as system performance is highly dependent on the computing and network conditions of a given deployment. However, the core logic of FIMAT, which primarily involves protocol conversion through basic IF-ELSE rules, introduces minimal time overhead. Similar latency figures for protocol conversion have been reported in [29], where the overhead was approximately 2 ms, whereas in our implementation, the measured latency ranges from 0.5 to 1 ms. Notably, the hardware specifications used in [29] were not explicitly stated, whereas our evaluation was conducted on a Raspberry Pi 4.

Regarding the overall performance of an interaction cycle, [35] reports that registering a Matter device requires approximately 781 ms, whereas in our implementation, the process is completed in approximately 422.3 ms. While the interaction time with Matter devices remains largely similar between both approaches, the higher delay observed in [35] is primarily attributed to the processing time of the oneM2M server. These variations are inherently dependent on the specific cloud platform implementation. In terms of bridging functionality, both approaches are comparable. Additionally, the overall time required for updating and processing commands in [35] is similar to that observed in our work.

8. Discussions

8.1. Scalability of the FIMAT

To address the concern regarding scalability, it is important to clarify that although a smart city may consist of millions of households, FIMAT is designed to operate at the scale of a single home. In a realistic smart-city architecture, each household deploys its own IoT Agent instance (whether embedded in a home gateway, router, or edge device). This aligns with FIWARE's recommended federated-edge design, where data processing and device management occur

locally, and only higher-level, aggregated context data is exposed to city platforms. Therefore, city-wide scalability does not require a single FIMAT instance to manage millions of devices. Instead, the system scales horizontally, one IoT agent per home, each responsible for its own Matter devices. Under this model, FIMAT only needs to scale to the upper bound of devices in a single residence. This deployment strategy ensures both practicality and scalability while preserving privacy and local autonomy.

We explicitly evaluated whether the current FIMAT architecture can support up to 1,000 Matter devices per residence, which we consider a realistic upper bound for future smart homes. The first scalability enabler comes directly from the Matter protocol itself. Starting from Matter 1.3, the introduction of batch command processing allows multiple device commands to be bundled into a single message. Prior to v1.3, controlling large groups of devices (e.g., 100 lights) resulted in the well-known "popcorn effect" due to sequential command dispatches. In contrast, batch processing enables near-simultaneous execution and drastically reduces both command latency and per-device control overhead. Our design fully exploits this capability, which is essential for scaling to hundreds or thousands of devices.

On the FIMAT side, our current implementation adopts a **thread-per-device** model using Node.js Worker Threads. The main thread remains lightweight and delegates device-specific tasks to dedicated workers. In our deployment on a Raspberry Pi 4 (with 4 GB RAM), where approximately 800 MB is reserved for the OS and background services, each worker thread requires less than 5 MB of memory. This allows us to support more than 500 concurrent threads (corresponding to 500 Matter nodes) on a Raspberry Pi with 4 GB of memory. Upgrading to an 8 GB Raspberry Pi 4 increases usable memory to approximately 7 GB, enabling well above 1,000 worker threads, which is sufficient for connecting smart homes to a smart city platform.

8.2. Identity, access control, and data governance considerations

While the primary focus of this work is on enabling interoperability between Matter devices and NGSI-based smart-city platforms, privacy, consent, and governance are equally critical for real-world deployment. Fig. 2 includes FIWARE security components because they provide a well-established foundation for addressing these concerns. Although a full security implementation is beyond the scope of this paper, we

outline here the recommended architecture and design directions that we consider suitable for ensuring the responsible use of household Matter data at the city scale.

8.2.1. Authentication and authorization

For authenticating smart-city applications and controlling access to household Matter data, FIWARE's standard identity and access-control suite provides a complete solution:

- Identity Management with **Keyrock**: for managing user, device, and application identities.
- **OAuth2** for delegated authorization.
- **Wilma PEP-Proxy** for validating access tokens and enforcing access policies at each API boundary

OAuth scopes or roles can be used to grant granular permissions, ensuring that city services receive only the level of access explicitly authorized by the homeowner.

8.2.2. Permission and policy management

For more fine-grained access control, FIWARE's Permission Manager (**Keypass**) offers a structured way to define and manage permissions associated with NGSI resources. Keypass enables:

- Attribute-level access permissions
- Explicit control over which applications may read/update specific NGSI properties
- Easy revocation or modification of permissions by the data owner

This fits naturally with the principle that the homeowner remains the primary data controller. Beyond static permissions, FIWARE's **Steel-skin** provides policy based access control (PBAC), allowing dynamic enforcement of governance rules. Security and privacy policies can be defined at a higher level and enforced consistently. This mechanism is particularly useful when different smart city applications require different levels of trust or data sensitivity.

8.2.3. Trusted data exchange

To address trust, provenance, and accountability in cross-organizational data flows, the FIWARE Trusted Data Exchange (FTC 1.0.0) component can be incorporated. FTC provides:

- Cryptographically verifiable data lineage
- End-to-end trust verification between data provider and consumer
- Audit logs that record when and how data is accessed
- Support for sharing sensitive data while maintaining strict governance policies

Given that household data may include behavioral information, FTC offers a valuable layer of transparency and accountability for smart-city ecosystems.

8.2.4. Owner consent, minimization, and retention

To balance the FAIR data principles with household privacy:

- Owner consent can be expressed via **OAuth** grants or dynamic policies in **Steelskin**
- Data minimization can be enforced by exposing only aggregated or abstracted indicators through the FIMAT
- Retention policies can be configured directly in the broker using per-attribute time-to-live, ensuring operational data does not persist longer than necessary.

These practices reduce privacy risks while retaining the operational and analytical value needed for smart city services.

8.3. Potential use case of the FIMAT

At the residential scale, the integration of Matter smart home appliances into the FIWARE smart city platform via the FIMAT holds significant potential for enhancing fire detection and prevention at the city level. Matter-enabled devices, such as smart smoke detectors, temperature sensors, and appliance controllers, can provide real-time, standardized data from homes and buildings across urban areas. By connecting this data to FIWARE's context management and interoperability framework, municipalities can aggregate and analyze environmental and safety information at scale. This integration allows for early detection of fire hazards, coordinated emergency responses, and predictive analytics to identify high-risk zones. Moreover, the seamless interoperability ensured by Matter's common data model and FIWARE's NGSI-based context interfaces can facilitate automated alerts to fire departments, infrastructure shutdowns (e.g., gas, electricity), and even real-time evacuation guidance, thereby enabling a proactive and city-wide fire prevention strategy.

At the city-wide level, significant efforts are underway in the European Union to promote sustainable energy systems, including the recent establishment of a Code of Conduct for Energy Smart Appliances.⁸ This initiative encourages device manufacturers to design appliances capable of shifting their electricity consumption based on user preferences and contextual parameters, thereby contributing to grid stability and potentially reducing household electricity costs. In response, many manufacturers and standard development organizations have begun aligning with these recommendations. Notably, with the release of Matter specification version 1.3, support for energy management clusters has been introduced. The integration of Matter-compliant appliances into the FIWARE smart city platform, as proposed in this work, directly supports this vision, enabling coordinated, scalable energy optimization across urban environments and paving the way toward a more sustainable and intelligent energy infrastructure for smart cities.

9. Concluding remarks

Matter has the potential to be a transformative smart home protocol, given its widespread adoption by major global consumer electronics manufacturers. This study presents a solution that integrates Matter with FIWARE, a well-established platform for smart city services. This integration facilitates the development of more advanced and sophisticated smart home-smart city applications, such as optimizing the management of sustainable urban electricity grids and enhancing disaster response efforts. By leveraging data from smart homes, it becomes possible to generate blackout maps or detect individuals who may be trapped inside their homes during emergencies—all while utilizing low-cost Matter devices.

We propose a FIWARE Smart Data Model for Matter, enabling applications and services to interact with Matter devices directly through FIWARE without requiring prior knowledge of the Matter specification. By leveraging the machine-readable specification introduced in Matter version 1.3, we implemented a programmable Matter specification, enabling the automatic generation of the Matter data model for FIWARE. This approach enhances the consistency of Matter specifications while promoting long-term interoperability with other cloud platforms. The generated data model fully covers the Matter specification as defined in version 1.4, the latest release. Moreover, it adheres to FIWARE Smart Data Model guidelines and, at the time of writing this article, is under review for official adoption within the FIWARE ecosystem.

We have implemented the FIWARE IoT Agent for Matter, extending the FIWARE generic enabler, the FIWARE IoT Agent. This agent follows standard procedures for integrating IoT devices into FIWARE, and in

⁸ <https://ses.jrc.ec.europa.eu/development-of-policy-proposals-for-energy-smart-appliances>

our work, we successfully integrated Matter into the FIWARE ecosystem. The implemented IoT Agent software was developed and deployed on an embedded hardware (the Raspberry Pi 4 model B), which serves as a Matter controller within a network of Matter devices. The proposed solution was evaluated using both commercial and experimental Matter devices, demonstrating its practical applicability in real smart home environments. The results indicate that the overhead introduced by protocol translation is negligible, and the overall interaction sequence is comparable to existing approaches, further validating its effectiveness.

Future work

As mentioned earlier, semantic interoperability is essential for fully leveraging the potential of the NGSI-LD data model proposed in this work. To achieve this, there is a clear need for an ontology that formally represents the semantics of Matter devices and services. As a future direction, we plan to develop a dedicated Matter ontology aligned with the SAREF ontology, which is widely adopted in IoT and energy-related domains. This ontology will serve as a semantic bridge, enabling consistent interpretation and integration of Matter data within FIWARE-based smart city systems. Furthermore, we aim to validate this integration through deployment in a real-world smart city testbed. In particular, we are looking forward to contributing to and testing our solution within the scope of ongoing European initiatives, such as the InterConnect⁹ project, which provides a collaborative environment for demonstrating interoperability and data-driven services in sustainable city infrastructures.

CRediT authorship contribution statement

Nguyen Thi Dieu Linh: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Conceptualization.
Thanh-Trung Nguyen: Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors extend their sincere gratitude to the anonymous reviewers for their valuable feedback and comments.

Appendix. Sample data

In this section, a sample of the smart plug data shown in Fig. A.18, including the following information:

- Raw data captured at the Matter interface: `tapoHADump.json`
- Decoded data at the Matter interface: `tapoHADump-Decoded.json`
- Generated NGSI-V2 resources: `ngsi-v2-normalized-tapo.json`
- Generated NGSI-LD resources: `ngsi-ld-normalized-tapo.jsonld`, and `ngsi-ld-normalized-profile.jsonld` (`context.jsonld` is currently empty).

The GitHub repository is available at:

<https://github.com/trungnguyen875452/FIMAT/tree/main/SampleData>



Fig. A.18. TP-Link smart plug Tapo L535E.

Data availability

Data will be made available on request.

References

- [1] A. Zanella, N. Bui, A. Castellani, L. Vangelista, M. Zorzi, Internet of things for smart cities, *IEEE Internet Things J.* 1 (1) (2014) 22–32, <http://dx.doi.org/10.1109/JIOT.2014.2306328>.
- [2] V.W.S. Chan, Internet of things and smart cities, *IEEE Commun. Mag.* 59 (10) (2021) 4–6, <http://dx.doi.org/10.1109/MCOM.2021.9627843>.
- [3] C. Goumopoulos, Smart city middleware: A survey and a conceptual framework, *IEEE Access* 12 (2024) 4015–4047, <http://dx.doi.org/10.1109/ACCESS.2023.3349376>.
- [4] F. Cirillo, G. Solmaz, E.L. Berz, M. Bauer, B. Cheng, E. Kovacs, A standard-based open source IoT platform: FIWARE, *IEEE Internet Things Mag.* 2 (3) (2019) 12–18, <http://dx.doi.org/10.1109/IOTM.0001.1800022>.
- [5] F. Cirillo, D. Gómez, L. Diez, I. Eliceigui Maestro, T.B.J. Gilbert, R. Akhavan, Smart city IoT services creation through large-scale collaboration, *IEEE Internet Things J.* 7 (6) (2020) 5267–5275, <http://dx.doi.org/10.1109/JIOT.2020.2978770>.
- [6] P. Fernández, J.M. Santana, S. Ortega, A. Trujillo, J.P. Suárez, C. Domínguez, J. Santana, A. Sánchez, SmartPort: A platform for sensor data monitoring in a seaport based on FIWARE, *Sensors* 16 (3) (2016) <http://dx.doi.org/10.3390/s16030417>, URL <https://www.mdpi.com/1424-8220/16/3/417>.
- [7] J. López-Riquelme, N. Pavón-Pulido, H. Navarro-Hellín, F. Soto-Valles, R. Torres-Sánchez, A software architecture based on FIWARE cloud for precision agriculture, *Agricult. Water. Manag.* 183 (2017) 123–135, <http://dx.doi.org/10.1016/j.agwat.2016.10.020>, Special Issue: Advances on ICTs for Water Management in Agriculture. URL <https://www.sciencedirect.com/science/article/pii/S0378377416304061>.
- [8] T. Panagiotakopoulos, D.P. Vlachos, T.V. Bakalakos, A. Kanavos, A. Kameas, A FIWARE-based IoT framework for smart water distribution management, in: 2021 12th International Conference on Information, Intelligence, Systems & Applications, IISA, 2021, pp. 1–6, <http://dx.doi.org/10.1109/IISA5424.2021.9555509>.
- [9] V.C. Pham, Y. Lim, Y. Tan, N.Y. Chong, Support for ECHONET-based smart home environments in the universal ecosystem, in: 2018 IEEE International Conference on Consumer Electronics, ICCE, 2018, pp. 1–4, <http://dx.doi.org/10.1109/ICCE.2018.8326218>.
- [10] G.P. Filho, L.A. Villas, V.P. Gonçalves, G. Pessin, A.A. Loureiro, J. Ueyama, Energy-efficient smart home systems: Infrastructure and decision-making process, *Internet Things* 5 (2019) 153–167, <http://dx.doi.org/10.1016/j.iot.2018.12.004>.
- [11] D. Marikyan, S. Papagiannidis, E. Alamanos, Smart home technology acceptance: An empirical investigation, in: I.O. Pappas, P. Mikalef, Y.K. Dwivedi, L. Jaccheri, J. Krogstie, M. Mäntymäki (Eds.), *Digital Transformation for a Sustainable Society in the 21st Century*, Springer International Publishing, Cham, 2019, pp. 305–315.
- [12] H. Sequeiros, T. Oliveira, M.A. Thomas, The impact of IoT smart home services on psychological well-being, *Inf. Syst. Front.* 24 (3) (2021) 1009–1026, <http://dx.doi.org/10.1007/s10796-021-10118-8>.
- [13] J. Dahmen, D.J. Cook, X. Wang, W. Honglei, Smart secure homes: a survey of smart home technologies that sense, assess, and respond to security threats, *J. Reliab. Intell. Environ.* 3 (2) (2017) 83–98, <http://dx.doi.org/10.1007/s40860-017-0035-0>.
- [14] W. Zegeye, A. Jemal, K. Kornegay, Connected smart home over matter protocol, in: 2023 IEEE International Conference on Consumer Electronics, ICCE, 2023, pp. 1–7, <http://dx.doi.org/10.1109/ICCE56470.2023.10043520>.
- [15] D. Belli, P. Barsocchi, F. Palumbo, Connectivity standards alliance matter: State of the art and opportunities, *Internet Things* 25 (2024) 101005, <http://dx.doi.org/10.1016/j.iot.2023.101005>.

⁹ <https://interconnectproject.eu/>

- [16] K. Shashwat, F. Hahn, X. Ou, A. Singhal, Security analysis of trust on the controller in the matter protocol specification, in: 2023 IEEE Conference on Communications and Network Security, CNS, 2023, pp. 1–6, <http://dx.doi.org/10.1109/CNS59707.2023.10288747>.
- [17] Requirements and architecture of the home energy management system and home network services, (Y.2070) 2015.
- [18] A. Luke, B. Ali, D.C. Renato, M. Lanfranco, R.H. Jaime, L. Kyong-Yul, T.-R. Bettina, V. Ugo, FIWARE4cities, (Edition 6) 2024.
- [19] S. Loss, H.P. Singh, N. Cacho, F. Lopes, Using FIWARE and blockchain in smart cities solutions, *Clust. Comput.* 26 (4) (2022) 2115–2128, <http://dx.doi.org/10.1007/s10586-022-03732-x>.
- [20] J.M.C. Fonseca, F.G. Marquez, T. Jacobs, FIWARE-NGSI v2 specification, Inf. Tec. V2. 0. FIWARE Found. (2018).
- [21] G. Privat, Guidelines for modeling with NGSI-LD, (1st Edition) 2021,
- [22] Connectivity Standards Alliance, Matter specification, 2024.
- [23] H.-S. Kim, S. Kumar, D.E. Culler, Thread/OpenThread: A compromise in low-power wireless multihop network architecture for the internet of things, *IEEE Commun. Mag.* 57 (7) (2019) 55–61, <http://dx.doi.org/10.1109/MCOM.2019.1800788>.
- [24] Connectivity Standards Alliance, Matter application cluster specification, 2024.
- [25] Connectivity Standards Alliance, Matter device library specification, 2024.
- [26] R.H. Pereira, Lightweight data bridge for connecting self-service end-user analytic tools to NGSI-based IoT systems, *Internet Things* 25 (2024) 101125, <http://dx.doi.org/10.1016/j.iot.2024.101125>.
- [27] P. Salhofer, J. Buchsbaum, M. Janusch, Building a smart city platform: A FIWARE example, in: *Smart Cities and Smart Governance*, Springer International Publishing, 2021, pp. 323–344, http://dx.doi.org/10.1007/978-3-030-61033-3_15.
- [28] S. Martins, R. Rossetti, Z. Kokkinogenis, Home automation system architecture based on FIWARE and multi-agent systems, 2022, <http://dx.doi.org/10.36227/techrxiv.19388513.v2>.
- [29] V.C. Pham, Y. Makino, Y. Tan, A FIWARE IoT agent for ECHONET lite protocol, in: 2020 IEEE 9th Global Conference on Consumer Electronics, GCCE, 2020, pp. 235–239, <http://dx.doi.org/10.1109/GCCE50665.2020.9291983>.
- [30] V.C. Pham, Y. Lim, A. Sgorbissa, Y. Tan, An ontology-driven ECHONET lite adaptation layer for smart homes, *J. Inf. Process.* 27 (2019) 360–368, <http://dx.doi.org/10.2197/ipsjip.27.360>.
- [31] V.C. Pham, T. Nguyen-Mau, M. Sioutis, Y. Tan, Enabling ECHONET lite devices in the matter ecosystem: A bridge solution, in: 2023 IEEE 12th Global Conference on Consumer Electronics, GCCE, 2023, pp. 867–868, <http://dx.doi.org/10.1109/GCCE59613.2023.10315429>.
- [32] S. Nagashige, K. Tanaka, H. Suzuki, Extension of iHAC hub specification for supporting interconnectivity between matter and ECHONET lite, in: 2022 IEEE 11th Global Conference on Consumer Electronics, GCCE, 2022, pp. 779–780.
- [33] M. Schenk, Matter and the Web of Things (Ph.D. thesis), Vienna University of Technology, Vienna, Austria, 2023.
- [34] L. Sciallo, L. Gigli, F. Montori, A. Trotta, M.D. Felice, A survey on the web of things, *IEEE Access* 10 (2022) 47570–47596, <http://dx.doi.org/10.1109/ACCESS.2022.3171575>.
- [35] D.L. Nguyen Thi, X.T. Kieu, T.S. Bui, T.L. Le, V.C. Pham, Towards interworking of matter and oneM2M: Design and implementation of a matter-oneM2M interworking proxy entity, *Internet Things* 27 (2024) 101313, <http://dx.doi.org/10.1016/j.iot.2024.101313>, URL <https://www.sciencedirect.com/science/article/pii/S2542660524002543>.
- [36] J. An, F. Le Gall, J. Kim, J. Yun, J. Hwang, M. Bauer, M. Zhao, J. Song, Toward global IoT-enabled smart cities interworking using adaptive semantic adapter, *IEEE Internet Things J.* 6 (3) (2019) 5753–5765, <http://dx.doi.org/10.1109/JIOT.2019.2905275>.
- [37] V.C. Pham, N.T.D. Linh, T. Le, T.H. Nguyen, Y. Tan, Automated data model generation from textual specifications: A case study of ECHONET lite specification, *IEEE Access* 11 (2023) 138316–138324, <http://dx.doi.org/10.1109/ACCESS.2023.3335595>.
- [38] R. Garcia-Castro, M. Lefrançois, M. Poveda-Villalon, L. Daniele, The ETSI SAREF ontology for smart applications: A long path of development and evolution, in: *Energy Smart Appliances: Applications, Methodologies, and Challenges*, 2023, pp. 183–215, <http://dx.doi.org/10.1002/9781119899457.ch7>.
- [39] K. Janowicz, A. Haller, S.J. Cox, D. Le Phuoc, M. Lefrançois, SOSA: A lightweight ontology for sensors, observations, samples, and actuators, *J. Web Semant.* 56 (2019) 1–10, <http://dx.doi.org/10.1016/j.websem.2018.06.003>, URL <https://www.sciencedirect.com/science/article/pii/S1570826818300295>.
- [40] M.D. Wilkinson, M. Dumontier, I.J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L.B. da Silva Santos, P.E. Bourne, J. Bouwman, A.J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C.T. Evelo, R. Finkers, A. Gonzalez-Beltran, A.J. Gray, P. Groth, C. Goble, J.S. Grethe, J. Heringa, P.A. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S.J. Lusher, M.E. Martone, A. Mons, A.L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M.A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR guiding principles for scientific data management and stewardship, *Sci. Data* 3 (1) (2016) <http://dx.doi.org/10.1038/sdata.2016.18>.
- [41] V.C. Pham, T. Nguyen-Mau, M. Sioutis, Y. Tan, Matter and ECHONET lite: Similarities, differences, and a bridge solution for interoperability, *Internet Things* 27 (2024) 101265, <http://dx.doi.org/10.1016/j.iot.2024.101265>, URL <https://www.sciencedirect.com/science/article/pii/S2542660524002063>.
- [42] S. Elamanov, H. Son, B. Flynn, S.K. Yoo, N. Dilshad, J. Song, Interworking between modbus and internet of things platform for industrial services, *Digit. Commun. Networks* (2022) <http://dx.doi.org/10.1016/j.dcan.2022.09.013>.
- [43] R.K. Lenka, S. Mamgain, S. Kumar, R.K. Barik, Performance analysis of automated testing tools: Jmeter and TestComplete, in: 2018 International Conference on Advances in Computing, Communication Control and Networking, ICACCCN, 2018, pp. 399–407, <http://dx.doi.org/10.1109/ICACCCN.2018.8748521>.
- [44] K. Kolehmainen, M. Pirazzi, J. Soininen, J. Backman, Simulation based performance evaluation of FIWARE IoT platform for smart agriculture, in: G. Wills, L. Buttyan, P. Kacsuk, V. Chang (Eds.), *Proceedings of the 8th International Conference on Internet of Things, Big Data and Security, IoTBDS 2023*, in: *International Conference on Internet of Things, Big Data and Security (IoTBDS)*, Science and Technology Publications, Lda, Portugal, 2023, pp. 73–81, <http://dx.doi.org/10.5220/0011918700003482>.